TRƯỜNG ĐẠI HỌC TRÀ VINH

KHOA KỸ THUẬT VÀ CÔNG NGHỆ

ĐỒ ÁN THỰC TẬP CƠ SỞ NGÀNH

XÂY DỰNG WEBSITE KẾT NỐI NGƯỜI TÌM VIỆC LÀM THÊM VỚI NHÀ TUYỂN DỤNG

Giảng viên hướng dẫn: **THS. TRẦM HOÀNG NAM**

Sinh viên thực hiện: **BÙI ANH KHOA**

Mã số sinh viên: **170123594**

Lớp: **DX23TT11**

Khoá: **2023–2027**

TP. Hồ Chí Minh, tháng 9 năm 2026

TRƯỜNG ĐẠI HỌC TRÀ VINH

KHOA KỸ THUẬT VÀ CÔNG NGHỆ

ĐỒ ÁN THỰC TẬP CƠ SỞ NGÀNH

XÂY DỰNG WEBSITE KẾT NỐI NGƯỜI TÌM VIỆC LÀM THÊM VỚI NHÀ TUYỂN DỤNG

Giảng viên hướng dẫn: **THS. TRẦM HOÀNG NAM**

Sinh viên thực hiện: **BÙI ANH KHOA**

Mã số sinh viên: **170123594**

Lớp: **DX23TT11**

Khoá: **2023–2027**

TP. Hồ Chí Minh, tháng 9 năm 2026

MỞ ĐẦU

1. Lý do chọn đề tài

Nhu cầu làm thêm của sinh viên và lao động thời vụ luôn ở mức cao: sinh viên cần thu nhập và kinh nghiệm thực tế song song với việc học, trong khi các quán ăn, cửa hàng, kho hàng và dịch vụ giao nhận thường xuyên thiếu nhân sự cho ca ngắn hạn, cuối tuần hoặc theo mùa vụ. Đây là mối quan hệ cung-cầu tồn tại thường trực nhưng lại chưa được kết nối hiệu quả bằng một kênh chuyên biệt.

Trên thực tế, việc tìm và đăng tin việc làm thêm hiện nay chủ yếu diễn ra qua các kênh phi chính thức như nhóm Facebook, Zalo hoặc truyền miệng. Cách làm này có ba hạn chế rõ rệt. Thứ nhất, tin tuyển dụng nằm rải rác trên nhiều nhóm khác nhau, khó tìm kiếm và dễ trôi mất sau vài giờ đăng. Thứ hai, các kênh này không có bộ lọc theo khu vực, khung giờ làm việc hay mức lương, buộc người tìm việc phải đọc thủ công từng bài đăng để chọn lọc thông tin phù hợp. Thứ ba, không có cơ chế xác thực người đăng tin, khiến tin tuyển dụng giả hoặc lừa đảo thu phí có cơ hội xuất hiện, đồng thời nhà tuyển dụng cũng khó theo dõi ai đã ứng tuyển vì phải nhắn tin riêng lẻ với từng người và dễ bỏ sót ứng viên phù hợp.

Trong khi đó, các nền tảng tuyển dụng trực tuyến quy mô lớn như TopCV, VietnamWorks lại tập trung vào việc làm toàn thời gian với quy trình đăng tin và hồ sơ ứng tuyển khá nặng, không phù hợp với đặc thù của một tin tuyển dụng làm thêm ngắn hạn cần đăng nhanh và tuyển gấp. Khoảng trống giữa hai nhóm kênh này — kênh phi chính thức thiếu công cụ, và nền tảng lớn thiếu sự phù hợp — là lý do đồ án chọn xây dựng một website chuyên biệt cho việc kết nối người tìm việc làm thêm với nhà tuyển dụng, tập trung vào đăng tin nhanh, tìm kiếm và lọc theo tiêu chí sát với nhu cầu thực tế của công việc bán thời gian, cùng cơ chế ứng tuyển và kiểm duyệt tin đơn giản, vừa đủ với quy mô của một đồ án cơ sở ngành.

2. Mục tiêu

2.1. Mục tiêu chung

Xây dựng một website kết nối người tìm việc làm thêm với nhà tuyển dụng, cho phép đăng tin, tìm kiếm, lọc và ứng tuyển việc làm bán thời gian một cách nhanh chóng, có quản lý tài khoản theo vai trò và trang quản trị kiểm duyệt tin đăng.

2.2. Mục tiêu cụ thể

- Khảo sát và phân tích yêu cầu của hai nhóm người dùng chính: người tìm việc làm thêm và nhà

tuyển dụng (cửa hàng, quán ăn, doanh nghiệp nhỏ).

- Thiết kế cơ sở dữ liệu quan hệ cho người dùng, tin tuyển dụng, hồ sơ ứng tuyển và thông báo.
- Xây dựng API RESTful cho các chức năng đăng tin, tìm kiếm và lọc, ứng tuyển, quản lý hồ sơ và

kiểm duyệt.

- Xây dựng giao diện web cho ba vai trò: ứng viên, nhà tuyển dụng và quản trị viên.
- Cài đặt chức năng tìm kiếm, lọc theo khu vực, khung giờ, mức lương và thông báo khi hồ sơ ứng

tuyển được phản hồi.

- Kiểm thử hệ thống theo các kịch bản nghiệp vụ chính và triển khai một phiên bản chạy được để

trình diễn.

Như vậy, đồ án đặt ra tổng cộng 6 mục tiêu cụ thể, cùng phục vụ một mục tiêu chung nêu trên.

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

- Người tìm việc làm thêm: chủ yếu là sinh viên và lao động phổ thông cần công việc bán thời

gian, theo ca hoặc ngắn hạn.

- Nhà tuyển dụng: cửa hàng, quán ăn, kho hàng, doanh nghiệp nhỏ có nhu cầu tuyển nhân sự thời

vụ hoặc bán thời gian.

- Kiến trúc và công nghệ xây dựng ứng dụng web: giao diện, API, cơ sở dữ liệu và cơ chế xác

thực người dùng.

3.2. Phạm vi và các lựa chọn của đồ án

Đồ án giới hạn phạm vi ở mức phù hợp với quy mô đồ án cơ sở ngành, thể hiện qua các lựa chọn có chủ đích sau:

Khía cạnh	**Lựa chọn của đồ án**	**Lý do**
Nền tảng	Website đáp ứng (responsive), dùng được trên cả máy tính và điện thoại	Không cần cài đặt, triển khai và cập nhật nhanh
Loại việc làm	Việc làm thêm, bán thời gian, theo ca hoặc thời vụ	Đúng nhu cầu thực tế của sinh viên và cửa hàng nhỏ
Vai trò người dùng	Ứng viên, nhà tuyển dụng, quản trị viên	Phân tách rõ quyền hạn và luồng thao tác
Xác thực	Đăng ký/đăng nhập bằng email và mật khẩu, phân	Đơn giản, đủ dùng cho quy

Khía cạnh	Lựa chọn của đồ án	Lý do
	quyền theo vai trò	mô đồ án
Kiểm duyệt	Quản trị viên duyệt tin trước khi hiển thị công khai	Hạn chế tin giả hoặc tin không phù hợp
Thông báo	Cập nhật trạng thái qua cơ chế truy vấn định kỳ (polling) trong ứng dụng, không dùng kênh ngoài	Đơn giản, không phụ thuộc dịch vụ email/SMS bên thứ ba

3.3. Những nội dung không thực hiện

- Ứng dụng di động (mobile app) riêng biệt; đồ án chỉ làm phiên bản web đáp ứng.
- Thanh toán lương hoặc ký hợp đồng lao động điện tử qua hệ thống.
- Chatbot hoặc gợi ý việc làm bằng trí tuệ nhân tạo; việc lọc và gợi ý dựa trên tiêu chí do

người dùng tự chọn.

- Xác minh danh tính bằng giấy tờ tùy thân hoặc tích hợp cổng thanh toán thật; hệ thống chỉ

dừng ở mức xác thực tài khoản cơ bản bằng email và mật khẩu.

4. Phương pháp thực hiện

Đồ án được xây dựng theo kiến trúc client-server, tách riêng giao diện người dùng (front-end) và máy chủ xử lý nghiệp vụ (back-end), giao tiếp với nhau qua API RESTful. Công nghệ cụ thể được lựa chọn và phiên bản đã dùng:

- Front-end: Next.js 16 (App Router) kết hợp TypeScript và TailwindCSS 3.
- Back-end: NestJS 11 (Node.js) kết hợp TypeScript, tổ chức theo module.
- Cơ sở dữ liệu: PostgreSQL, thao tác qua Prisma ORM 6.

- Xác thực: JSON Web Token (JWT) kết hợp mã hoá mật khẩu bằng bcrypt và kiểm soát truy cập theo

vai trò (role-based access control).

- Kiểm thử: Jest cho kiểm thử đơn vị và tích hợp phía back-end, Testing Library cho front-end.
- Đóng gói và triển khai bản demo: Docker Compose.

Phiên bản các công nghệ trên được cố định lùi lại một bản chính so với bản phát hành mới nhất tại thời điểm xây dựng (ví dụ Prisma dùng bản 6 thay vì bản 7 mới đổi cách cấu hình datasource, NestJS dùng bản 11 thay vì bản 12 chỉ phát hành dưới dạng ESM chưa tương thích thẳng với công cụ kiểm thử ts-jest), nhằm giữ các quy ước cấu hình còn phổ biến trong tài liệu hướng dẫn và tránh các thay đổi phá vỡ tương thích (breaking changes) mới phát sinh, mà không ảnh hưởng đến tính năng nào của hệ thống.

Quy trình phát triển áp dụng cách tiếp cận lặp: từng nhóm chức năng (xác thực, đăng tin, tìm kiếm, ứng tuyển, kiểm duyệt, thông báo) được thiết kế cơ sở dữ liệu, cài đặt API, cài đặt giao diện và kiểm thử ngay sau khi hoàn thành, thay vì dồn toàn bộ việc kiểm thử về giai đoạn cuối.

5. Bố cục báo cáo

Báo cáo gồm năm chương, theo đúng trình tự quy định của khoa:

- **Chương 1 — Tổng quan**: khảo sát hiện trạng tìm việc làm thêm hiện nay, phân tích các nền

tảng liên quan và khoảng trống thị trường mà đồ án hướng tới giải quyết.

- **Chương 2 — Nghiên cứu lý thuyết**: trình bày cơ sở lý thuyết và các công cụ, công nghệ được

sử dụng để xây dựng hệ thống.

- **Chương 3 — Hiện thực hóa nghiên cứu**: mô tả kiến trúc, thiết kế cơ sở dữ liệu, thiết kế

API, thiết kế giao diện và quá trình cài đặt từng chức năng.

- **Chương 4 — Kết quả nghiên cứu**: trình bày kết quả kiểm thử, đối chiếu với các chỉ tiêu đã

đề ra và một số đánh giá về giao diện, hiệu năng.

- **Chương 5 — Kết luận và hướng phát triển**: tổng kết những gì đã đạt được và đề xuất hướng mở

rộng cho đồ án.

LỜI CẢM ƠN

Em xin chân thành cảm ơn ThS. Trầm Hoàng Nam, giảng viên hướng dẫn, đã tận tình chỉ dẫn định hướng đề tài, góp ý về kiến trúc hệ thống và cách trình bày báo cáo trong suốt quá trình thực hiện đồ án.

Em cũng xin cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Khoa Kỹ thuật và Công nghệ, Trường Đại học Trà Vinh đã truyền đạt kiến thức nền tảng làm cơ sở để em hoàn thành đồ án này.

Do thời gian thực hiện có hạn và kinh nghiệm thực tế còn hạn chế, đồ án khó tránh khỏi thiếu sót. Em mong nhận được góp ý của quý thầy cô để hoàn thiện hơn.

NHẬN XÉT

(Của giảng viên hướng dẫn)

……

……

……

……

……

……

……

……

……

……

Giảng viên hướng dẫn

(ký và ghi rõ họ tên)

BẢN NHẬN XÉT ĐỒ ÁN THỰC TẬP CƠ SỞ NGÀNH

(Của giảng viên hướng dẫn)

UBND TỈNH TRÀ VINH — CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

TRƯỜNG ĐẠI HỌC TRÀ VINH — Độc lập – Tự do – Hạnh Phúc

Họ và tên sinh viên: Bùi Anh Khoa. MSSV: 170123594.

Ngành: Công nghệ Thông tin. Khóa: 2023–2027.

Tên đồ án: Xây dựng website kết nối người tìm việc làm thêm với nhà tuyển dụng.

Họ và tên Giảng viên hướng dẫn: ThS. Trầm Hoàng Nam.

Chức danh: ……. Học vị: Thạc sĩ.

NHẬN XÉT

1. Nội dung đồ án

……

……

……

……

2. Ưu điểm

……

……

3. Khuyết điểm

……

……

4. Điểm mới đồ án

……

……

5. Giá trị thực tế trên đồ án

……

……

6. Đề nghị sửa chữa bổ sung

……

……

7. Đánh giá

……

……

Trà Vinh, ngày …… tháng …… năm 20……

Giảng viên hướng dẫn

(Ký & ghi rõ họ tên)

NHẬN XÉT

(Của giảng viên chấm)

……

……

……

……

……

……

……

……

Giảng viên chấm

(ký và ghi rõ họ tên)

BẢN NHẬN XÉT ĐỒ ÁN THỰC TẬP CƠ SỞ NGÀNH

(Của cán bộ chấm đồ án)

UBND TỈNH TRÀ VINH — CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

TRƯỜNG ĐẠI HỌC TRÀ VINH — Độc lập - Tự do - Hạnh phúc

Họ và tên người nhận xét:

……

Chức danh: ……. Học vị:

…….

Chuyên ngành:

……

Cơ quan công tác:

……

Tên sinh viên: Bùi Anh Khoa.

Tên đề tài đồ án: Xây dựng website kết nối người tìm việc làm thêm với nhà tuyển dụng.

I. Ý KIẾN NHẬN XÉT

1. Nội dung

……

……

……

……

2. Điểm mới các kết quả của đồ án

……

……

3. Ứng dụng thực tế

……

……

II. CÁC VẤN ĐỀ CẦN LÀM RÕ

(Các câu hỏi của giáo viên phản biện)

……

……

……

……

……

……

III. KẾT LUẬN

(Ghi rõ đồng ý hay không đồng ý cho bảo vệ đồ án)

……

……

……, ngày …… tháng …… năm 20……

Người nhận xét

(Ký & ghi rõ họ tên)

MỤC LỤC

MỞ ĐẦU.....

1. Lý do chọn đề tài.....

2. Mục tiêu.....

2.1. Mục tiêu chung.....

2.2. Mục tiêu cụ thể.....

3. Đối tượng và phạm vi nghiên cứu.....

3.1. Đối tượng nghiên cứu.....

3.2. Phạm vi và các lựa chọn của đồ án.....

3.3. Những nội dung không thực hiện.....

4. Phương pháp thực hiện.....

5. Bố cục báo cáo.....

CHƯƠNG 1. TỔNG QUAN.....1

1.1. Khảo sát hiện trạng.....1

1.2. Các nền tảng tuyển dụng phổ biến hiện nay.....1

1.3. Các kênh phi chính thức.....1

1.4. So sánh và khoảng trống thị trường.....1

1.5. Hướng tiếp cận của đồ án.....2

CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT.....3

2.1. Kiến trúc ứng dụng web client-server.....3

2.2. Giao thức và kiến trúc API RESTful.....3

2.3. Framework front-end và back-end.....3

2.4. Cơ sở dữ liệu quan hệ và ORM.....4

2.5. Xác thực JWT, băm mật khẩu và kiểm soát truy cập theo vai trò.....4

2.5.1. Xác thực bằng JSON Web Token....4
2.5.2. Băm mật khẩu bằng bcrypt....5
2.5.3. Kiểm soát truy cập theo vai trò (RBAC)....5
2.6. Cơ chế cập nhật thông báo: polling thay vì WebSocket....5
2.7. Kiểm tra hợp lệ dữ liệu đầu vào....6
2.8. Nguyên tắc chuẩn hóa dữ liệu quan hệ và ràng buộc toàn vẹn....7
2.9. Kiểm thử phần mềm: kiểm thử đơn vị và kiểm thử tích hợp....8
2.10. REST so với các kiểu kiến trúc API khác....9
2.11. Bảo mật ứng dụng web: đối chiếu với OWASP Top 10....9
CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU....12
3.1. Kiến trúc tổng quan hệ thống....12
3.2. Thiết kế cơ sở dữ liệu....12
3.3. Thiết kế API....17
3.4. Luồng ứng tuyển chi tiết....20
3.5. Cài đặt xác thực và phân quyền....22
3.6. Cài đặt từng use case....22
3.6.1. UC01 — Đăng ký / đăng nhập tài khoản....22
3.6.2. UC02 — Đăng tin tuyển dụng....22
3.6.3. UC03 — Tìm kiếm và lọc tin tuyển dụng....23
3.6.4. UC04 — Ứng tuyển vào tin tuyển dụng....25
3.6.5. UC05 — Quản lý danh sách ứng viên đã ứng tuyển....26
3.6.6. UC06 — Duyệt hoặc từ chối tin tuyển dụng....27
3.6.7. UC07 — Nhận thông báo cập nhật hồ sơ....28
3.7. Giao diện khác....29

3.8. Xử lý lỗi và kiểm tra hợp lệ đầu vào.....30
CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU.....32
4.1. Kết quả kiểm thử tự động.....32
4.2. Đối chiếu với chỉ tiêu định lượng đã đề ra.....32
4.3. Một số nhận xét về giao diện.....33
4.4. Hạn chế đã biết.....33
4.5. Phạm vi kiểm thử theo nhóm chức năng.....34
4.6. Đánh giá khả năng mở rộng hệ thống.....36
4.7. Đối chiếu kết quả với hiện trạng ban đầu.....36
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....38
5.1. Kết luận.....38
5.2. Hướng phát triển.....38
DANH MỤC TÀI LIỆU THAM KHẢO.....40
PHỤ LỤC A. TOÀN VĂN SCHEMA CƠ SỞ DỮ LIỆU.....42
PHỤ LỤC B. TOÀN VĂN DANH SÁCH API.....45

DANH MỤC CÁC BẢNG, HÌNH

Bảng

Bảng 1.1. So sánh các kênh tìm việc làm thêm hiện có.....2

Bảng 2.1. Đối chiếu với các nhóm rủi ro liên quan trong OWASP Top 10:2021.....10

Bảng 3.1. Bảng `User`.....13

Bảng 3.2. Bảng `Job`.....14

Bảng 3.3. Bảng `Application`.....15

Bảng 3.4. Bảng `Profile`.....15

Bảng 3.5. Bảng `Notification`.....16

Bảng 3.6. Danh sách API chính theo module.....17

Bảng 4.1. Đối chiếu chỉ tiêu định lượng.....32

Bảng 4.2. Đối chiếu use case với bộ kiểm thử tự động.....34

Bảng 4.3. Đối chiếu hạn chế ban đầu với giải pháp đã cài đặt.....37

Hình

Hình 2.1. Kiến trúc tổng quan của hệ thống Việc Làm Thêm.....12

Hình 2.2. Sơ đồ quan hệ thực thể (ERD) của hệ thống, trích trực tiếp từ `backend/prisma/schema.prisma`.....12

Hình 2.3. Sơ đồ tuần tự luồng ứng tuyển đầu-cuối.....20

Hình 3.1. Form đăng tin.....22

Hình 1.1. Trang chủ với danh sách tin phù hợp.....23

Hình 1.2. Trang tìm việc với bộ lọc đầy đủ.....24

Hình 1.3. Trang chi tiết một tin tuyển dụng, nơi ứng viên bấm ứng tuyển.....25

Hình 3.2. Trang quản lý ứng viên với hai hồ sơ ở hai trạng thái khác nhau.....26

Hình 3.3. Trang quản trị với tổng quan hệ thống, tin chờ duyệt và danh sách người dùng 27

Hình 3.4. Chuông thông báo đang mở, hiển thị một thông báo chưa đọc.....28

Hình 3.5. Trang hồ sơ cá nhân của ứng viên (`/ho-so`), nơi ứng viên xem lại thông tin tài khoản của mình.....29

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

- API: Application Programming Interface
- GVHD: Giảng viên hướng dẫn
- JWT: JSON Web Token
- MSSV: Mã số sinh viên
- ORM: Object-Relational Mapping
- RBAC: Role-Based Access Control (Kiểm soát truy cập theo vai trò)
- REST: Representational State Transfer
- SVTH: Sinh viên thực hiện

TÓM TẮT

Nhu cầu tìm việc làm thêm của sinh viên và lao động thời vụ luôn ở mức cao, trong khi các kênh hiện có (nhóm mạng xã hội hoặc các nền tảng tuyển dụng lớn) đều không phù hợp: kênh phi chính thức thiếu công cụ tìm kiếm và dễ trôi tin, còn các nền tảng lớn tập trung vào việc làm toàn thời gian với quy trình nặng. Đồ án xây dựng một website kết nối người tìm việc làm thêm với nhà tuyển dụng, tập trung vào đăng tin nhanh, tìm kiếm và lọc theo khu vực/khung giờ/mức lương, cùng cơ chế ứng tuyển và kiểm duyệt tin đơn giản, phù hợp quy mô một đồ án cơ sở ngành.

Hệ thống có ba vai trò — ứng viên, nhà tuyển dụng, quản trị viên — xây dựng trên Next.js (front-end), NestJS và PostgreSQL/Prisma (back-end), xác thực bằng JWT kết hợp phân quyền theo vai trò. Toàn bộ 7 use case đã đặc tả (đăng ký/đăng nhập, đăng tin, tìm kiếm/lọc, ứng tuyển, quản lý ứng viên, duyệt tin, nhận thông báo) đều đã cài đặt và kiểm thử.

Kết quả đạt được: một hệ thống chạy được đầy đủ ba vai trò, đóng gói bằng Docker Compose để triển khai demo, cùng bộ kiểm thử tự động cho các luồng nghiệp vụ chính. Báo cáo trình bày chi tiết cơ sở lý thuyết, quá trình phân tích - thiết kế - cài đặt, và đánh giá kết quả đối chiếu với các chỉ tiêu đã đề ra.

CHƯƠNG 1. TỔNG QUAN

1.1. Khảo sát hiện trạng

Thị trường tuyển dụng trực tuyến tại Việt Nam hiện có nhiều nền tảng hoạt động, nhưng phần lớn không tập trung vào việc làm thêm ngắn hạn, linh hoạt theo ca. Bên cạnh đó, một phần lớn tin tuyển dụng làm thêm vẫn được đăng qua các kênh mạng xã hội không có công cụ hỗ trợ tìm kiếm. Mục này khảo sát cả hai nhóm kênh nói trên để làm rõ khoảng trống mà đồ án hướng tới lấp đầy.

1.2. Các nền tảng tuyển dụng phổ biến hiện nay

TopCV [15] và VietnamWorks [17] là hai nền tảng tuyển dụng trực tuyến lớn tại Việt Nam, mạnh về việc làm toàn thời gian với hồ sơ ứng viên (CV) chi tiết, quy trình đăng tin và duyệt hồ sơ khá đầy đủ nhưng cũng vì vậy mà khá nặng đối với một quán ăn hay cửa hàng nhỏ cần tuyển gấp nhân sự cho một ca làm việc. JobStreet [7] có phạm vi hoạt động ở nhiều quốc gia Đông Nam Á, giao diện tìm kiếm phong phú nhưng chưa tối ưu bộ lọc theo khung giờ làm việc linh hoạt kiểu bán thời gian. Ở một số thị trường nước ngoài, các nền tảng chuyên biệt cho việc làm thời vụ như Instawork [6] và Wonolo [18] cho thấy rõ hơn xu hướng tách riêng thị trường việc làm thời vụ, theo ca khỏi thị trường việc làm toàn thời gian — hiển thị trực tiếp mức lương và khung giờ ngay trên danh sách tin, cho phép ứng tuyển nhanh trong một thao tác.

1.3. Các kênh phi chính thức

Phần lớn tin tuyển dụng làm thêm tại Việt Nam vẫn được đăng trên các nhóm Facebook dạng "Việc làm thêm sinh viên", các kênh Zalo, hoặc truyền miệng giữa sinh viên với nhau. Ưu điểm của cách làm này là miễn phí và dễ đăng tin ngay lập tức. Nhược điểm là tin trôi rất nhanh trong dòng thời gian của nhóm, không có công cụ tìm kiếm hay lọc theo tiêu chí, và khó kiểm chứng độ tin cậy của người đăng tin.

1.4. So sánh và khoảng trống thị trường

Bảng 1 tổng hợp ưu điểm và hạn chế của ba nhóm kênh hiện có, đối chiếu với hướng tiếp cận của đồ án.

Bảng 1.1. So sánh các kênh tìm việc làm thêm hiện có

Kênh	**Ưu điểm**	**Hạn chế**
Nền tảng tuyển dụng lớn (TopCV, VietnamWorks)	Uy tín, hồ sơ ứng viên đầy đủ, nhiều nhà tuyển dụng lớn	Quy trình nặng, không tối ưu cho tin tuyển gấp, ca ngắn
Nhóm Facebook, Zalo	Miễn phí, đăng tin nhanh, tiếp cận cộng đồng sinh viên	Không tìm kiếm/lọc được, tin trôi nhanh, khó xác thực
Website của đồ án	Tập trung vào việc làm thêm, lọc theo ca/khu vực/lương, có kiểm duyệt	Phạm vi demo còn giới hạn, chưa có lượng người dùng thật

Có thể thấy hai nhóm kênh hiện có nằm ở hai thái cực: nền tảng lớn thì đầy đủ nhưng nặng, kênh phi chính thức thì nhanh nhưng thiếu công cụ. Khoảng trống nằm ở giữa hai thái cực này — một kênh vừa đủ nhẹ để đăng tin nhanh, vừa có công cụ tìm kiếm/lọc và một bước kiểm duyệt tối thiểu để hạn chế tin giả.

1.5. Hướng tiếp cận của đồ án

Từ khoảng trống trên, đồ án xây dựng một website chuyên biệt cho việc làm thêm: đăng tin nhanh gọn, tìm kiếm và lọc theo khu vực, khung giờ và mức lương, có bước kiểm duyệt tin của quản trị viên để giảm rủi ro tin giả, và một luồng ứng tuyển đơn giản giúp nhà tuyển dụng theo dõi ứng viên tập trung tại một nơi thay vì nhắn tin rời rạc trên nhiều kênh. Chi tiết cơ sở lý thuyết và công nghệ dùng để hiện thực hoá hướng tiếp cận này được trình bày ở Chương 2; chi tiết thiết kế và cài đặt được trình bày ở Chương 3.

CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT

2.1. Kiến trúc ứng dụng web client-server

Ứng dụng web hiện đại thường được tách thành ba phần: giao diện người dùng (client) chạy trên trình duyệt, máy chủ ứng dụng (server) xử lý nghiệp vụ, và cơ sở dữ liệu lưu trữ thông tin. Client gọi đến server thông qua API, server truy vấn cơ sở dữ liệu rồi trả kết quả về dưới dạng dữ liệu (thường ở định dạng JSON) để client hiển thị. Cách tách lớp này cho phép phát triển, kiểm thử và mở rộng từng phần độc lập. Đây cũng chính là kiến trúc mà hệ thống Việc Làm Thêm áp dụng, với Next.js đóng vai trò client và NestJS đóng vai trò server, giao tiếp qua REST API.

Trong hai mô hình render phổ biến của một ứng dụng client-server (render phía máy chủ — SSR, server-side rendering — và render phía client, CSR, client-side rendering), Next.js App Router cho phép kết hợp cả hai trong cùng một dự án: trang chủ (danh sách tin tuyển dụng) được render phía máy chủ để tải nhanh và thân thiện với công cụ tìm kiếm, trong khi các thao tác tương tác (lọc, ứng tuyển, cập nhật trạng thái) xử lý phía client sau khi trang đã tải [10].

2.2. Giao thức và kiến trúc API RESTful

REST (Representational State Transfer) là kiểu kiến trúc API sử dụng các phương thức HTTP chuẩn (GET, POST, PATCH, DELETE) để thao tác trên tài nguyên [2]. Mục 2.10 so sánh REST với hai kiểu kiến trúc API khác (GraphQL, gRPC) để làm rõ lý do chọn REST cho đồ án này. Mỗi loại tài nguyên trong hệ thống, gồm tin tuyển dụng, hồ sơ ứng tuyển và thông báo, có một nhóm endpoint riêng, ví dụ `GET /jobs` để lấy danh sách tin, `POST /applications` để tạo một hồ sơ ứng tuyển mới, `PATCH /applications/:id` để nhà tuyển dụng cập nhật trạng thái hồ sơ. Dữ liệu trao đổi ở định dạng JSON, giúp API rõ ràng, dễ kiểm thử bằng công cụ như Postman hoặc Supertest, và dễ tái sử dụng cho nhiều loại giao diện sau này nếu hệ thống mở rộng sang ứng dụng di động.

2.3. Framework front-end và back-end

Next.js [10] (dựa trên React) được chọn cho front-end vì hỗ trợ định tuyến theo cấu trúc thư mục (App Router), tách trang thành các route rõ ràng như đã cài đặt trong dự án

(`/viec-lam`, `/nha-tuyen-dung`, `/quan-tri`), đồng thời hỗ trợ render phía máy chủ giúp trang chủ tải nhanh.

NestJS [9] (dựa trên Node.js) được chọn cho back-end vì tổ chức mã nguồn theo module rõ ràng: mỗi nhóm chức năng của hệ thống (xác thực, tin tuyển dụng, hồ sơ ứng tuyển, thông báo, quản trị) là một module riêng gồm controller, service và, khi cần, guard. Cơ chế dependency injection sẵn có giúp tái sử dụng logic, ví dụ `RolesGuard` áp dụng lại cho nhiều controller khác nhau mà không phải khởi tạo lại từng nơi.

2.4. Cơ sở dữ liệu quan hệ và ORM

Cơ sở dữ liệu quan hệ PostgreSQL [12] tổ chức dữ liệu thành các bảng có quan hệ khóa ngoại với nhau. Trong hệ thống này là 5 bảng `User`, `Job`, `Application`, `Profile`, `Notification`, với quan hệ ví dụ một `Job` thuộc về một `User` có vai trò nhà tuyển dụng, và một `Application` liên kết một `User` có vai trò ứng viên với một `Job` cụ thể. Prisma ORM [13] cho phép thao tác với các bảng này bằng mã lệnh hướng đối tượng (`prisma.job.findMany(...)`) thay vì viết trực tiếp câu lệnh SQL, đồng thời sinh ra các migration theo dõi lịch sử thay đổi cấu trúc bảng, giúp giảm lỗi cú pháp và dễ bảo trì khi mô hình dữ liệu thay đổi. Mục 2.8 trình bày thêm nguyên tắc chuẩn hóa và các ràng buộc toàn vẹn áp dụng khi thiết kế 5 bảng này.

2.5. Xác thực JWT, băm mật khẩu và kiểm soát truy cập theo vai trò

2.5.1. Xác thực bằng JSON Web Token

Hệ thống dùng JSON Web Token (JWT) [8] để xác thực người dùng sau khi đăng nhập thành công. Một JWT gồm ba phần nối bằng dấu chấm:

$$\text{JWT} = \text{base64url}(\text{header}) \;.\; \text{base64url}(\text{payload}) \;.\; \text{base64url}(\text{signature})$$

trong đó `header` khai báo thuật toán ký (hệ thống dùng HMAC-SHA256), `payload` chứa thông tin người dùng cần thiết cho việc phân quyền (`sub` là id người dùng, `role` là vai trò), và `signature` được máy chủ ký bằng khóa bí mật để chống giả mạo. Sau khi đăng nhập, client đính kèm token này trong header `Authorization` của mỗi yêu cầu tiếp

theo để chứng minh danh tính mà không cần gửi lại mật khẩu. Trong cấu hình thật của hệ thống, token có hiệu lực 7 ngày (`JWT_EXPIRES_IN=7d`).

2.5.2. Băm mật khẩu bằng bcrypt

Mật khẩu người dùng không được lưu ở dạng văn bản thuần mà được băm bằng bcrypt [14] trước khi lưu vào cột `passwordHash`. bcrypt là một hàm băm một chiều có tích hợp salt ngẫu nhiên và một hệ số công việc (cost factor) điều chỉnh được, sao cho số vòng lặp tính toán tăng theo cấp số nhân với cost factor:

$$\text{số vòng lặp} = 2^{\text{cost}}$$

Hệ thống dùng cost factor bằng 10 (`BCRYPT_SALT_ROUNDS = 10` trong `backend/src/users/users.service.ts`), tức $2^{10} = 1024$ vòng lặp cho mỗi lần băm. Mức này đủ chậm để gây khó khăn cho tấn công dò mật khẩu hàng loạt, nhưng vẫn đủ nhanh để không ảnh hưởng trải nghiệm đăng ký/đăng nhập của người dùng thật.

2.5.3. Kiểm soát truy cập theo vai trò (RBAC)

Hệ thống có ba vai trò cố định trong cột `role` của bảng `User`: `CANDIDATE` (ứng viên), `EMPLOYER` (nhà tuyển dụng), `ADMIN` (quản trị viên). Việc kiểm soát truy cập theo vai trò (role-based access control — RBAC) [11] được cài đặt bằng một decorator tùy chỉnh `@Roles(...)` đánh dấu vai trò được phép trên từng route, kết hợp với một `RolesGuard` đọc lại metadata đó bằng `Reflector` của NestJS: nếu vai trò của người dùng đang đăng nhập không nằm trong danh sách được phép, guard trả về lỗi 403 (`ForbiddenException`) trước khi yêu cầu chạm tới logic nghiệp vụ. Cách làm này tách rõ luật phân quyền ra khỏi từng hàm xử lý, chỉ cần khai báo một dòng `@Roles(Role.ADMIN)` trên route thay vì kiểm tra vai trò thủ công trong mỗi hàm.

2.6. Cơ chế cập nhật thông báo: polling thay vì WebSocket

Khi có sự kiện cần thông báo (có ứng viên mới nộp hồ sơ, hồ sơ được cập nhật trạng thái), hệ thống cần đưa thông tin đó tới đúng người dùng đang mở ứng dụng. Có hai hướng kỹ thuật phổ biến để làm việc này. Một là WebSocket [1], một giao thức giữ kết nối hai chiều liên tục giữa client và server để server có thể chủ động đẩy dữ liệu ngay khi

sự kiện xảy ra. Hai là polling, trong đó client tự động gọi lại API theo chu kỳ cố định để hỏi xem có gì mới không.

Hệ thống Việc Làm Thêm chọn phương án polling: phía front-end gọi định kỳ `GET /notifications/mine` và `GET /notifications/mine/unread-count` mỗi 20 giây (cài đặt trong `frontend/src/lib/use-polling-notifications.ts`) để lấy số thông báo chưa đọc và danh sách thông báo mới. Lý do chọn polling thay vì WebSocket là để giữ toàn bộ giao tiếp trong cùng một giao thức REST đã dùng cho mọi chức năng khác, không phải duy trì thêm một kết nối máy chủ riêng (WebSocket gateway) chỉ để phục vụ một tính năng, phù hợp với quy mô một đồ án cơ sở ngành, đổi lại độ trễ thông báo tối đa khoảng 20 giây thay vì tức thời.

2.7. Kiểm tra hợp lệ dữ liệu đầu vào

Một nguyên tắc nền tảng của bảo mật ứng dụng web là không bao giờ tin tưởng dữ liệu gửi lên từ phía client, dù dữ liệu đó đã được kiểm tra ở giao diện người dùng [11]. Giao diện có thể bị bỏ qua hoàn toàn nếu người dùng gọi thẳng vào API bằng công cụ như curl hoặc Postman thay vì đi qua form, nên việc kiểm tra tính hợp lệ của dữ liệu đầu vào phải được thực hiện lại, độc lập, ở phía máy chủ trước khi dữ liệu đó được dùng để tạo hoặc cập nhật bản ghi trong cơ sở dữ liệu.

Hệ thống cài đặt việc kiểm tra này bằng thư viện `class-validator` [16], kết hợp với cơ chế Data Transfer Object (DTO) sẵn có của NestJS: mỗi endpoint nhận dữ liệu đầu vào khai báo một lớp DTO riêng, gắn decorator kiểm tra trên từng trường. Ví dụ `CreateJobDto` (`backend/src/jobs/dto/create-job.dto.ts`) yêu cầu `title` là chuỗi dài tối thiểu 3 ký tự (`@IsString() @MinLength(3)`), `description` dài tối thiểu 10 ký tự, còn `salaryMin`/`salaryMax` phải là số nguyên không âm (`@IsInt() @Min(0)`) nếu có khai báo. `CreateApplicationDto` (`backend/src/applications/dto/create-application.dto.ts`) yêu cầu `jobId` đúng định dạng UUID hợp lệ (`@IsUUID()`) — ràng buộc này từng chặn đúng một tình huống thật trong giai đoạn kiểm thử thủ công: một bộ dữ liệu mẫu cũ dùng ID dạng `00000000-0000-0000-0000-00000000000N` (không phải UUID hợp lệ ở bất kỳ phiên bản nào) bị `@IsUUID()` từ chối ngay ở bước xác thực, trước khi chạm tới logic

nghiệp vụ — đúng hành vi mong muốn của một lớp kiểm tra đầu vào, dù nguyên nhân gốc lúc đó là dữ liệu mẫu sai định dạng chứ không phải lỗi của validator.

Toàn bộ DTO được áp dụng tự động nhờ một `ValidationPipe` toàn cục khai báo trong `backend/src/main.ts`, bật ba tuỳ chọn: `whitelist: true` (tự động loại bỏ mọi trường không được khai báo trong DTO khỏi dữ liệu đầu vào, chặn trường hợp client cố gửi thêm trường không mong muốn như `role` hay `status` để ghi đè giá trị lẽ ra do server tự quyết định), `forbidNonWhitelisted: true` (từ chối toàn bộ yêu cầu với lỗi 400 nếu phát hiện trường lạ, thay vì âm thầm bỏ qua), và `transform: true` (tự động chuyển kiểu dữ liệu thô từ query string, ví dụ chuỗi `"15000"` thành số nguyên `15000`, trước khi so khớp với decorator `@IsInt()`). Cách làm này tập trung toàn bộ luật kiểm tra dữ liệu vào một nơi (lớp DTO), tránh lặp lại logic kiểm tra thủ công ở từng controller.

2.8. Nguyên tắc chuẩn hóa dữ liệu quan hệ và ràng buộc toàn vẹn

Thiết kế một cơ sở dữ liệu quan hệ tốt thường tuân theo các dạng chuẩn hóa (normal forms) nhằm giảm dư thừa dữ liệu và tránh các bất thường khi cập nhật (update anomaly) [12]. Dạng chuẩn thứ nhất (1NF) yêu cầu mỗi cột chỉ chứa một giá trị nguyên tố; dạng chuẩn thứ hai và thứ ba (2NF, 3NF) yêu cầu các cột không khóa phụ thuộc đầy đủ và trực tiếp vào khóa chính, không phụ thuộc bắc cầu qua một cột không khóa khác.

Áp dụng vào 5 bảng của hệ thống (mục 3.2): thông tin hồ sơ cá nhân (`bio`, `skills`, `preferredAreas`, `avatarUrl`) được tách thành một bảng `Profile` riêng thay vì gộp chung vào bảng `User`, vì đây là dữ liệu phụ thuộc vào một người dùng cụ thể nhưng không phải thuộc tính định danh tài khoản — tách bảng giúp bảng `User` gọn, chỉ chứa thông tin xác thực/định danh, đồng thời cho phép quan hệ 1-1 rõ ràng qua khóa ngoại `userId` có ràng buộc duy nhất (unique). Tương tự, một hồ sơ ứng tuyển (`Application`) không lưu lặp lại thông tin của tin tuyển dụng hay của ứng viên — nó chỉ lưu hai khóa ngoại (`jobId`, `candidateId`) cùng trạng thái riêng của chính hồ sơ ứng tuyển đó (`status`, `message`); thông tin đầy đủ về tin hay về ứng viên được lấy lại bằng cách join ngược về bảng `Job`/`User` qua Prisma khi cần hiển thị, tránh tình trạng dữ liệu bị lệch nhau giữa nhiều bản sao nếu ứng viên đổi tên hoặc nhà tuyển dụng sửa tin sau khi đã có người ứng tuyển.

Bên cạnh chuẩn hóa, hệ thống dùng thêm hai loại ràng buộc toàn vẹn do chính PostgreSQL đảm bảo, không phải do tầng ứng dụng tự kiểm tra: ràng buộc khóa ngoại (foreign key constraint), đảm bảo không thể tạo một `Application` trỏ tới một `jobId` hoặc `candidateId` không tồn tại; và ràng buộc duy nhất phức hợp (composite unique constraint) trên cặp `(jobId, candidateId)` của bảng `Application`, đảm bảo ở cấp cơ sở dữ liệu rằng một ứng viên không thể ứng tuyển hai lần vào cùng một tin, kể cả khi có lỗi logic ở tầng ứng dụng vô tình bỏ sót bước kiểm tra này.

2.9. Kiểm thử phần mềm: kiểm thử đơn vị và kiểm thử tích hợp

Kiểm thử tự động thường được tổ chức theo mô hình "kim tự tháp kiểm thử" (test pyramid) [3]: lớp đáy, chiếm số lượng nhiều nhất, là kiểm thử đơn vị (unit test) — kiểm tra một hàm hoặc một lớp độc lập, cô lập khỏi các phụ thuộc bên ngoài (cơ sở dữ liệu, mạng) bằng cách thay thế các phụ thuộc đó bằng đối tượng giả lập (mock); lớp giữa là kiểm thử tích hợp (integration test) — kiểm tra nhiều thành phần phối hợp với nhau, ví dụ một request HTTP thật đi qua toàn bộ pipeline của framework (guard, pipe, controller, service) rồi chạm tới cơ sở dữ liệu thật; lớp đỉnh, chiếm số lượng ít nhất, là kiểm thử đầu-cuối (end-to-end test) — mô phỏng thao tác người dùng thật trên giao diện. Mô hình này khuyến nghị nhiều kiểm thử đơn vị vì chạy nhanh và dễ xác định chính xác vị trí lỗi, còn kiểm thử tích hợp/đầu-cuối tuy chậm hơn nhưng xác nhận được rằng các thành phần thực sự phối hợp đúng với nhau trong điều kiện gần giống môi trường thật.

Hệ thống Việc Làm Thêm áp dụng hai lớp đầu của mô hình này ở phía back-end bằng Jest. Kiểm thử đơn vị (các file `*.service.spec.ts` và `roles.guard.spec.ts` trong `backend/src/`) kiểm tra logic nghiệp vụ của từng service một cách cô lập, dùng một bản giả lập của `PrismaClient` thay vì kết nối cơ sở dữ liệu thật, giúp bộ kiểm thử này chạy trong vài giây. Kiểm thử tích hợp (các file `*.e2e-spec.ts` trong `backend/test/`) dùng `supertest` để gửi request HTTP thật tới một instance Nest thật, chạy trên cơ sở dữ liệu kiểm thử riêng (`TEST_DATABASE_URL`, tách biệt với cơ sở dữ liệu phát triển), qua đó xác nhận toàn bộ chuỗi guard → pipe → controller → service → Prisma → PostgreSQL hoạt động đúng với nhau, không chỉ từng mảnh riêng lẻ. Kết quả thật của cả

hai lớp này được trình bày ở mục 4.1; mục 4.5 phân tích thêm phạm vi bao phủ theo từng nhóm chức năng.

2.10. REST so với các kiểu kiến trúc API khác

Ngoài REST, hai kiểu kiến trúc API phổ biến khác là GraphQL [4] và gRPC [5]. GraphQL cho phép client tự mô tả chính xác những trường dữ liệu mình cần trong một truy vấn duy nhất, phù hợp với ứng dụng có nhiều màn hình cần kết hợp dữ liệu từ nhiều nguồn khác nhau theo nhiều cách khác nhau, nhưng đòi hỏi xây dựng thêm một tầng schema/resolver và cơ chế kiểm soát truy cập phức tạp hơn vì mọi truy vấn đều đi qua một endpoint duy nhất thay vì nhiều endpoint tách theo tài nguyên như REST. gRPC dùng Protocol Buffers để tuần tự hóa dữ liệu ở dạng nhị phân thay vì JSON văn bản, cho tốc độ và băng thông tốt hơn REST, phù hợp cho giao tiếp giữa các dịch vụ nội bộ (service-to- service) trong kiến trúc vi dịch vụ, nhưng không được trình duyệt hỗ trợ gọi trực tiếp (cần một tầng gateway chuyển đổi) và khó debug thủ công hơn vì dữ liệu không phải dạng văn bản đọc được.

Với quy mô một đồ án cơ sở ngành có một front-end, một back-end nguyên khối (monolith, không tách vi dịch vụ) và tập tài nguyên tương đối ít và ổn định (5 loại), REST [2] là lựa chọn phù hợp nhất trong ba kiểu kiến trúc trên: không cần tầng schema/resolver bổ sung như GraphQL, tận dụng trực tiếp được các khái niệm HTTP chuẩn (mã trạng thái, cache theo URL, middleware xác thực theo route) mà cả NestJS lẫn các công cụ kiểm thử (curl, Postman, Supertest) đều hỗ trợ sẵn, và dữ liệu dạng JSON dễ đọc trực tiếp khi gỡ lỗi — điều có giá trị thực tế trong một dự án một người phát triển, nơi việc gỡ lỗi nhanh quan trọng hơn việc tối ưu băng thông ở quy mô vi dịch vụ.

2.11. Bảo mật ứng dụng web: đối chiếu với OWASP Top 10

OWASP Top 10 [11] là danh sách 10 nhóm rủi ro bảo mật phổ biến nhất ở ứng dụng web, do tổ chức phi lợi nhuận OWASP tổng hợp định kỳ từ dữ liệu thực tế của nhiều hệ thống. Bảng 2.1 đối chiếu từng nhóm rủi ro liên quan trực tiếp tới một hệ thống quy mô như Việc Làm Thêm với biện pháp đã cài đặt thật trong mã nguồn, nêu rõ cả những nhóm CHƯA có biện pháp (hạn chế thật, không che giấu) thay vì chỉ liệt kê các nhóm đã xử lý.

Bảng 2.1. Đối chiếu với các nhóm rủi ro liên quan trong OWASP Top 10:2021

Nhóm rủi ro	**Biện pháp đã cài đặt**	**Trạng thái**
A01 — Kiểm soát truy cập bị phá vỡ (Broken Access Control)	`RolesGuard` (mục 2.5.3) và `JobOwnerGuard` (mục 3.3) chặn truy cập sai vai trò hoặc sai chủ sở hữu ở tầng route, xác nhận bằng `rbac.e2e-spec.ts`	Đã có
A02 — Lỗi mật mã (Cryptographic Failures)	Mật khẩu băm bằng bcrypt (mục 2.5.2), không lưu văn bản thuần; JWT ký bằng HMAC-SHA256	Đã có
A03 — Chèn mã độc (Injection)	Prisma sinh câu lệnh SQL tham số hóa (parameterized query) cho mọi truy vấn, không ghép chuỗi SQL thủ công ở bất kỳ đâu trong mã nguồn — loại bỏ hoàn toàn khả năng SQL injection qua đường này	Đã có
A04 — Thiết kế thiếu an toàn (Insecure Design)	`ValidationPipe` toàn cục (mục 2.7) từ chối dữ liệu đầu vào không hợp lệ trước khi tới logic nghiệp vụ	Đã có
A05 — Cấu hình sai bảo mật (Security Misconfiguration)	`app.enableCors()` (`backend/src/main.ts`) bật CORS không giới hạn	Một phần — hạn chế biết trước

Nhóm rủi ro	**Biện pháp đã cài đặt**	**Trạng thái**
	origin cụ thể — phù hợp môi trường demo cho một đồ án nhưng cần giới hạn lại đúng domain thật nếu triển khai sản xuất	
A07 — Lỗi xác thực và nhận dạng (Identification and Authentication Failures)	JWT có thời hạn 7 ngày (mục 2.5.1); hệ thống hiện CHƯA giới hạn số lần thử đăng nhập sai (rate limiting) nên về lý thuyết vẫn có thể bị dò mật khẩu hàng loạt nếu kẻ tấn công gửi rất nhiều yêu cầu	Một phần — hạn chế biết trước

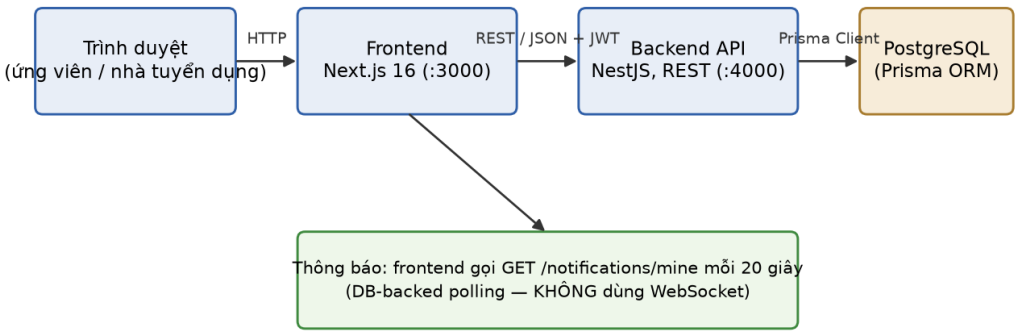
Hai dòng đánh dấu "một phần" không phải sai sót bị bỏ sót mà là đánh đổi phạm vi có chủ đích ở quy mô một đồ án cơ sở ngành: cấu hình CORS mở và chưa có rate limiting đều là những bổ sung có thể thêm sau (một middleware như `@nestjs/throttler` cho rate limiting, một danh sách origin cụ thể cho CORS) mà không cần thay đổi kiến trúc hiện tại — được ghi nhận lại ở mục "Hướng phát triển" (mục 5.2) thay vì cài đặt vội trong phạm vi đồ án này.

CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Kiến trúc tổng quan hệ thống

Hình 2.1 thể hiện kiến trúc tổng quan của hệ thống Việc Làm Thêm.

Hình 2.1. Kiến trúc tổng quan hệ thống Việc Làm Thêm



Hình 2.1 thể hiện kiến trúc tổng quan của hệ thống Việc Làm Thêm.

Nguồn: tự vẽ dựa trên docker-compose.yml và frontend/src/lib/use-polling-notifications.ts

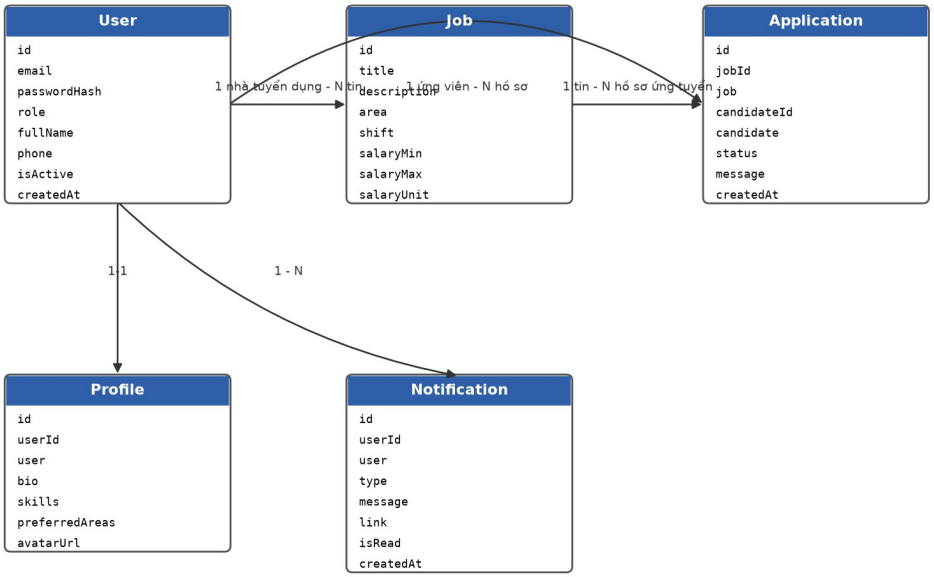
Hệ thống gồm ba thành phần triển khai độc lập, đóng gói bằng Docker Compose: dịch vụ `db` (PostgreSQL 16), dịch vụ `backend` (NestJS, lắng nghe cổng 4000) và dịch vụ `frontend` (Next.js, lắng nghe cổng 3000). Trình duyệt của người dùng chỉ giao tiếp trực tiếp với `frontend`; các thao tác cần dữ liệu (đăng nhập, tìm kiếm, ứng tuyển...) được `frontend` gọi tiếp sang `backend` qua REST API có JWT đính kèm. `backend` là thành phần duy nhất truy cập cơ sở dữ liệu, thông qua Prisma Client.

Riêng chức năng thông báo không dùng kết nối riêng: `frontend` gọi định kỳ `GET /notifications/mine` và `GET /notifications/mine/unread-count` mỗi 20 giây (lý do chọn cơ chế này thay vì WebSocket đã trình bày ở mục 2.6).

3.2. Thiết kế cơ sở dữ liệu

Hình 2.2 thể hiện sơ đồ quan hệ thực thể (ERD) của hệ thống, trích trực tiếp từ `backend/prisma/schema.prisma`.

Hình 2.2. Sơ đồ quan hệ thực thể (ERD) — trích trực tiếp từ backend/prisma/schema.prisma



Hình 2.2 thể hiện sơ đồ quan hệ thực thể (ERD) của hệ thống, trích trực tiếp từ `backend/prisma/schema.prisma`.

Nguồn: tự vẽ dựa trên backend/prisma/schema.prisma

Cơ sở dữ liệu gồm 5 bảng. Bảng 3.1 đến Bảng 3.5 mô tả chi tiết từng bảng, copy nguyên trạng tên field và kiểu dữ liệu từ `schema.prisma`, không diễn giải lại theo suy đoán.

Bảng 3.1. Bảng `User`

Field	**Kiểu dữ liệu**	**Ràng buộc**
id	String	Khóa chính, mặc định uuid()
email	String	Duy nhất (unique)
passwordHash	String	Bắt buộc — lưu mật khẩu đã băm bcrypt, không lưu văn bản thuần
role	Role (enum)	`CANDIDATE`,

Field	**Kiểu dữ liệu**	**Ràng buộc**
		`EMPLOYER`, hoặc `ADMIN`
fullName	String	Bắt buộc
phone	String	Tùy chọn
isActive	Boolean	Mặc định `true`
createdAt / updatedAt	DateTime	Tự động gán

Bảng 3.2. Bảng `Job`

Field	**Kiểu dữ liệu**	**Ràng buộc**
id	String	Khóa chính
title, description, area, shift	String	Bắt buộc
salaryMin, salaryMax	Int	Tùy chọn
salaryUnit	String	Mặc định `"VND/giờ"`
requirements	String	Tùy chọn
status	JobStatus (enum)	`PENDING`, `APPROVED`, hoặc `REJECTED`; mặc định `PENDING`
rejectReason	String	Tùy chọn — lý do khi admin từ chối

Field	**Kiểu dữ liệu**	**Ràng buộc**
employerId	String	Khóa ngoại tới `User` (quan hệ `EmployerJobs`)

Bảng 3.3. Bảng `Application`

Field	**Kiểu dữ liệu**	**Ràng buộc**
id	String	Khóa chính
jobId	String	Khóa ngoại tới `Job`
candidateId	String	Khóa ngoại tới `User`
status	ApplicationStatus (enum)	`PENDING`, `VIEWED`, `INTERVIEW`, hoặc `REJECTED`; mặc định `PENDING`
message	String	Tùy chọn
—	—	Ràng buộc duy nhất trên cặp `(jobId, candidateId)` — một ứng viên chỉ ứng tuyển một lần cho một tin

Bảng 3.4. Bảng `Profile`

Field	**Kiểu dữ liệu**	**Ràng buộc**
id	String	Khóa chính

Field	**Kiểu dữ liệu**	**Ràng buộc**
userId	String	Khóa ngoại tới `User`, duy nhất (quan hệ 1-1)
bio	String	Tùy chọn
skills	String[]	Mặc định mảng rỗng
preferredAreas	String[]	Mặc định mảng rỗng
avatarUrl	String	Tùy chọn

Bảng 3.5. Bảng `Notification`

Field	**Kiểu dữ liệu**	**Ràng buộc**
id	String	Khóa chính
userId	String	Khóa ngoại tới `User`
type	String	Loại thông báo, ví dụ `NEW_APPLICATION`
message	String	Nội dung hiển thị
link	String	Tùy chọn — đường dẫn điều hướng khi bấm vào
isRead	Boolean	Mặc định `false`
createdAt	DateTime	Tự động gán

3.3. Thiết kế API

Bảng 3.6 liệt kê các endpoint chính, đối chiếu trực tiếp với các file controller thật trong `backend/src/`.

Bảng 3.6. Danh sách API chính theo module

Phương thức	**Endpoint**	**Vai trò yêu cầu**	**Mô tả**
POST	`/auth/register`	Công khai	Đăng ký tài khoản (ứng viên hoặc nhà tuyển dụng)
POST	`/auth/login`	Công khai	Đăng nhập, trả về JWT
GET	`/auth/me`	Đã đăng nhập	Lấy thông tin tài khoản đang đăng nhập
GET	`/jobs`	Công khai	Tìm kiếm/lọc tin theo `q`, `area`, `shift`, `salaryMin`, `salaryMax`, phân trang `page`/`limit`
GET	`/jobs/:id`	Công khai	Xem chi tiết một tin
POST	`/jobs`	EMPLOYER	Đăng tin mới (trạng thái khởi tạo `PENDING`)
GET	`/jobs/mine`	EMPLOYER	Danh sách tin của chính nhà tuyển dụng đang đăng

Phương thức	**Endpoint**	**Vai trò yêu cầu**	**Mô tả**
			nhập
PATCH	`/jobs/:id`	EMPLOYER (chủ tin)	Sửa tin — có thêm `JobOwnerGuard` kiểm tra đúng chủ tin
DELETE	`/jobs/:id`	EMPLOYER (chủ tin)	Xoá tin — cùng `JobOwnerGuard`
POST	`/applications`	CANDIDATE	Ứng tuyển vào một tin
GET	`/applications/mine`	CANDIDATE	Danh sách hồ sơ đã ứng tuyển của ứng viên
GET	`/jobs/:id/ applications`	EMPLOYER (chủ tin)	Danh sách ứng viên của một tin — cùng `JobOwnerGuard`
PATCH	`/applications/:id/ status`	EMPLOYER	Cập nhật trạng thái hồ sơ (VIEWED/INTER VIEW/REJECTED)
GET	`/notifications/ mine`	Đã đăng nhập	Danh sách thông báo của người dùng
GET	`/notifications/ mine/unread-count`	Đã đăng nhập	Số thông báo chưa đọc

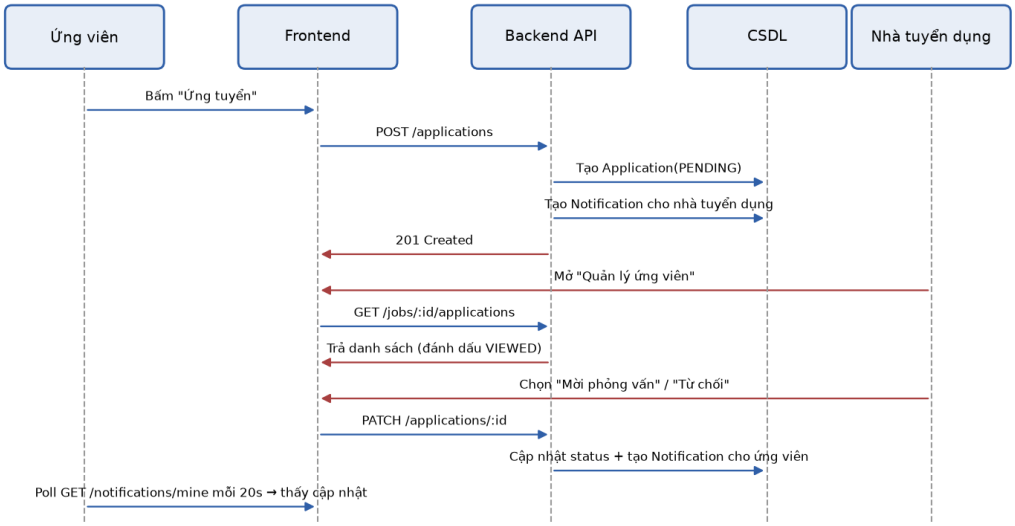
Phương thức	**Endpoint**	**Vai trò yêu cầu**	**Mô tả**
PATCH	`/notifications/:id/read`	Đã đăng nhập	Đánh dấu một thông báo đã đọc
PATCH	`/notifications/read-all`	Đã đăng nhập	Đánh dấu tất cả đã đọc
GET	`/profiles/me`	CANDIDATE	Lấy hồ sơ cá nhân
PATCH	`/profiles/me`	CANDIDATE	Cập nhật hồ sơ cá nhân
GET	`/admin/jobs`	ADMIN	Danh sách tin chờ duyệt
PATCH	`/admin/jobs/:id/approve`	ADMIN	Duyệt tin
PATCH	`/admin/jobs/:id/reject`	ADMIN	Từ chối tin
GET	`/admin/users`	ADMIN	Danh sách người dùng
GET	`/admin/stats`	ADMIN	Số liệu tổng quan cho trang quản trị

Ngoài `RolesGuard` mô tả ở mục 2.5.3, hai route sửa/xoá tin và xem danh sách ứng viên còn được bảo vệ thêm bởi `JobOwnerGuard` (`backend/src/jobs/guards/job-owner.guard.ts`) để đảm bảo một nhà tuyển dụng chỉ thao tác được trên tin do chính mình đăng, không thao tác được trên tin của nhà tuyển dụng khác dù cùng vai trò `EMPLOYER`.

3.4. Luồng ứng tuyển chi tiết

Hình 2.3 thể hiện sơ đồ tuần tự luồng ứng tuyển đầu-cuối.

Hình 2.3. Sơ đồ tuần tự luồng ứng tuyển đầu-cuối



Hình 2.3 thể hiện sơ đồ tuần tự luồng ứng tuyển đầu-cuối.

Nguồn: tự vẽ dựa trên luồng nghiệp vụ cài đặt trong backend/src/applications/

Luồng gồm các bước sau, đúng theo hợp đồng luồng (Apply-Flow Contract) đã xác lập từ giai đoạn thiết kế hệ thống:

1. Ứng viên tìm kiếm/lọc tin, mở trang chi tiết, bấm "Ứng tuyển".
2. Frontend gọi `POST /applications`; backend tạo một bản ghi `Application` với

`status = PENDING`, đồng thời tạo một `Notification` gửi cho nhà tuyển dụng sở hữu tin đó.

1. Nhà tuyển dụng mở trang "Quản lý ứng viên" (`GET /jobs/:id/applications`); lần xem đầu tiên

này được đánh dấu chuyển trạng thái ứng dụng sang `VIEWED`.

1. Nhà tuyển dụng chọn "Mời phỏng vấn" hoặc "Từ chối", frontend gọi `PATCH

/applications/:id/status`; backend cập nhật `status` thành `INTERVIEW` hoặc `REJECTED`, đồng thời tạo một `Notification` gửi cho ứng viên.

1. Ứng viên thấy cập nhật thông qua vòng poll `GET /notifications/mine` tiếp theo (tối đa 20

giây sau khi nhà tuyển dụng thao tác).

Chi tiết cài đặt từng use case, kèm ảnh chụp giao diện thật, được trình bày ở mục tiếp theo (`04b-chuong-3-cai-dat.md`).

3.5. Cài đặt xác thực và phân quyền

Việc đăng ký chỉ chấp nhận vai trò `CANDIDATE` hoặc `EMPLOYER` (`backend/src/auth/dto/register.dto.ts`); tài khoản `ADMIN` không tự đăng ký được mà chỉ được tạo sẵn qua script khởi tạo dữ liệu (`backend/prisma/seed.ts`). Mỗi route cần đăng nhập gắn `@UseGuards(JwtAuthGuard)` để xác thực token, và route cần giới hạn vai trò gắn thêm `@UseGuards(RolesGuard)` cùng decorator `@Roles(...)` như đã trình bày ở mục 2.5.3. Chương 3 chỉ nêu lại cách áp dụng thật, không lặp lại lý thuyết.

3.6. Cài đặt từng use case

3.6.1. UC01 — Đăng ký / đăng nhập tài khoản

Người dùng chọn vai trò khi đăng ký tại `/dang-ky` (chỉ giữa ứng viên và nhà tuyển dụng), sau đó đăng nhập tại `/dang-nhap`. Đăng nhập thành công trả về JWT và thông tin tài khoản, được lưu vào `localStorage` phía trình duyệt (`frontend/src/lib/auth-context.tsx`, khoá lưu trữ `vlt_auth`) để duy trì phiên đăng nhập giữa các lần tải trang.

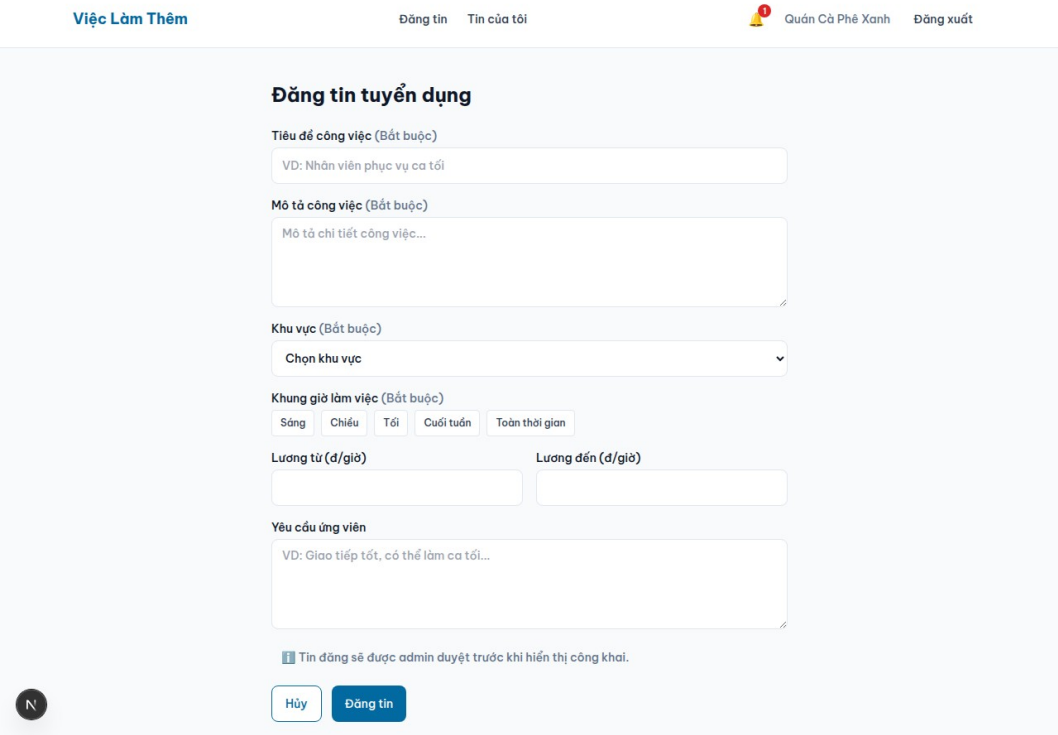
3.6.2. UC02 — Đăng tin tuyển dụng

Nhà tuyển dụng điền form tại `/nha-tuyen-dung/dang-tin`. Khi gửi, backend tạo bản ghi `Job` với `status` khởi tạo luôn là `PENDING`, không phụ thuộc dữ liệu client gửi lên:

```
// backend/src/jobs/jobs.service.ts:13-23
async create(employerId: string, dto: CreateJobDto): Promise<Job> {
  this.assertSalaryRange(dto.salaryMin, dto.salaryMax);
  return this.prisma.job.create({
    data: { ...dto, employerId, status: 'PENDING' },
  });
}
```

Hình 3.1 thể hiện form đăng tin.

Website kết nối việc làm thêm



Hình 3.1 thể hiện form đăng tin.

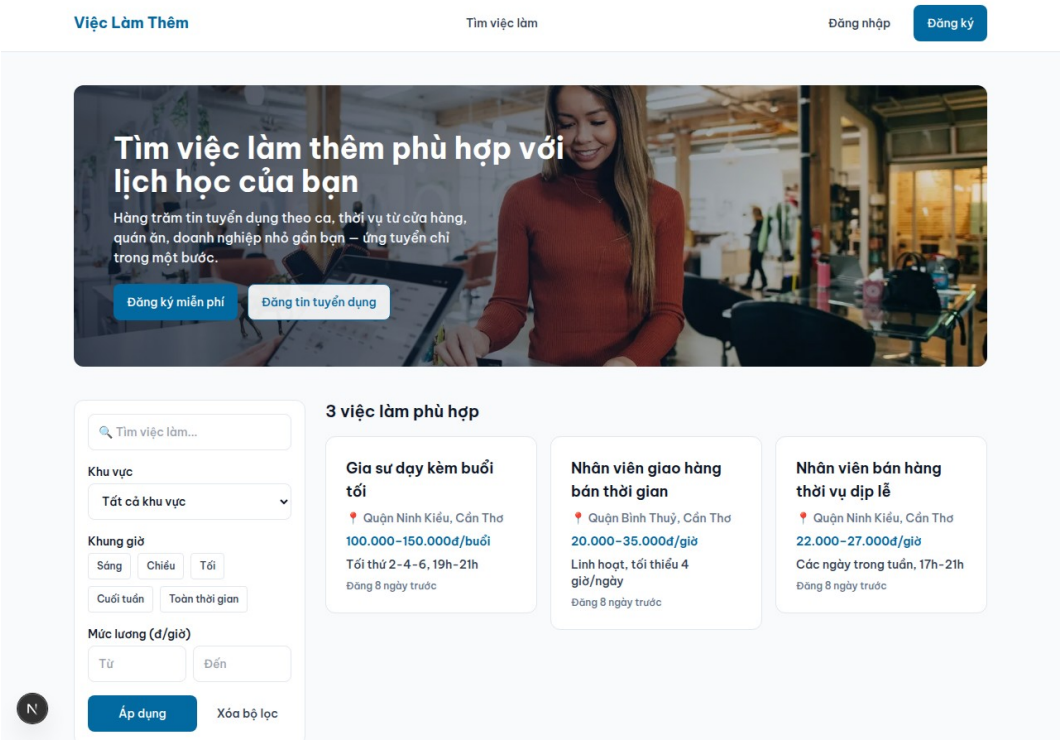
Nguồn: chụp từ hệ thống

3.6.3. UC03 — Tìm kiếm và lọc tin tuyển dụng

Trang chủ và trang `/viec-lam` cùng dùng một API `GET /jobs` với các tham số lọc `q` (từ khoá), `area` (khu vực), `shift` (khung giờ), `salaryMin`/`salaryMax` (khoảng lương), và phân trang `page`/`limit` (`backend/src/jobs/dto/list-jobs.dto.ts`). Chỉ tin có `status = APPROVED` được trả về cho người dùng công khai.

Hình 1.1 thể hiện trang chủ với danh sách tin phù hợp.

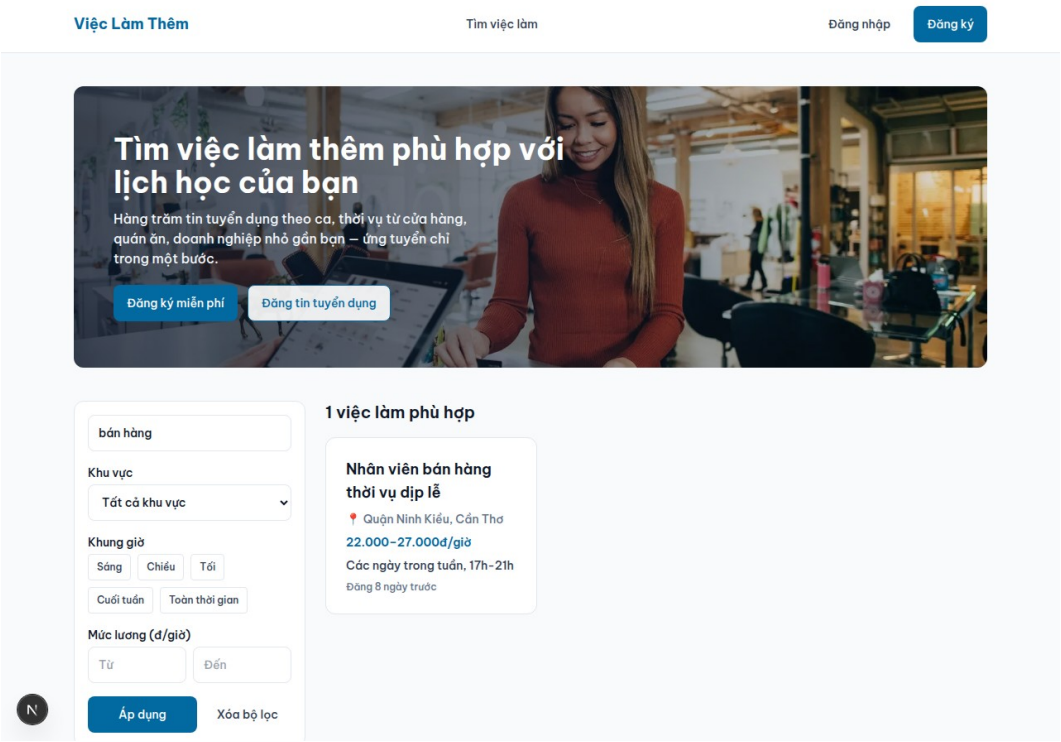
Website kết nối việc làm thêm



Hình 1.1 thể hiện trang chủ với danh sách tin phù hợp.

Nguồn: chụp từ hệ thống

Hình 1.2 thể hiện trang tìm việc với bộ lọc đầy đủ.



Hình 1.2 thể hiện trang tìm việc với bộ lọc đầy đủ.

Nguồn: chụp từ hệ thống

3.6.4. UC04 — Ứng tuyển vào tin tuyển dụng

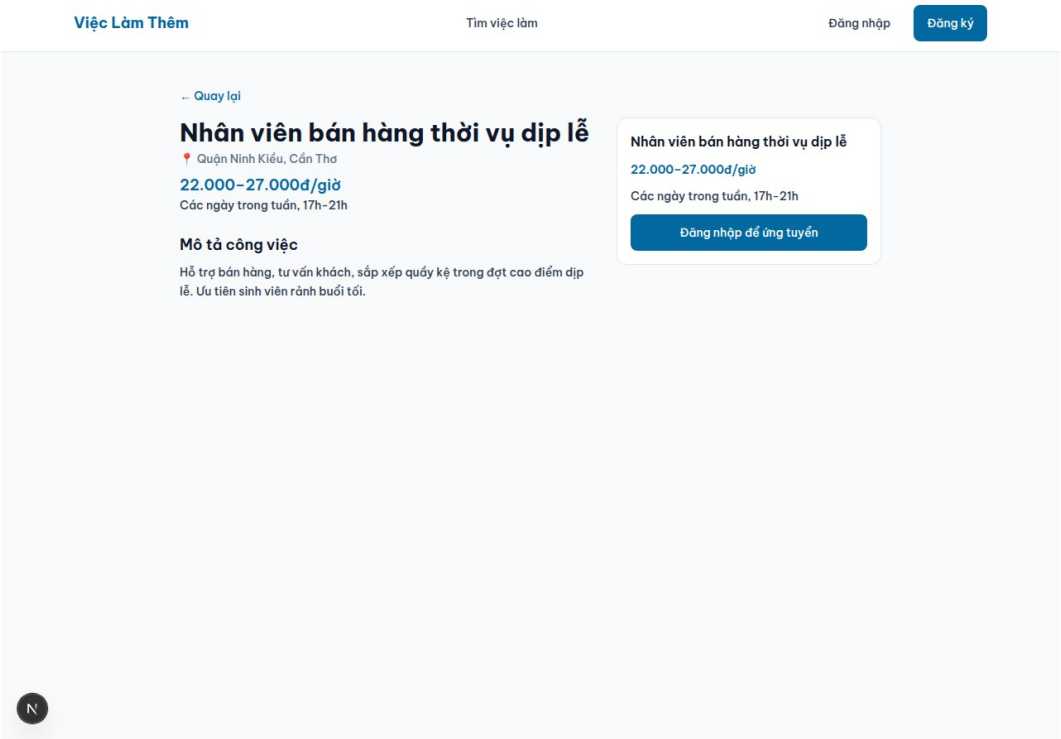
Từ trang chi tiết tin, ứng viên bấm "Ứng tuyển" để gọi `POST /applications`. Backend kiểm tra tin còn ở trạng thái `APPROVED` mới cho ứng tuyển, tạo bản ghi `Application`, và tạo ngay một `Notification` cho nhà tuyển dụng:

```
// backend/src/applications/applications.service.ts:28-57 (rút gọn)
async create(candidateId: string, dto: CreateApplicationDto) {
  const job = await this.prisma.job.findUnique({ where: { id: dto.jobId } });
  if (!job || job.status !== 'APPROVED') {
    throw new ConflictException('Tin tuyển dụng không còn nhận hồ sơ ứng
tuyển');
  }
  const application = await this.prisma.application.create({
    data: { jobId: dto.jobId, candidateId, message: dto.message },
  });
  await this.notificationsService.create({
    userId: job.employerId,
    type: 'APPLICATION_CREATED',
    message: `Có ứng viên mới ứng tuyển vào "${job.title}"`,
    link: `/nha-tuyen-dung/tin/${job.id}/ung-vien`,
  });
  return application;
}
```

Ràng buộc duy nhất `(jobId, candidateId)` ở cấp cơ sở dữ liệu (mục 3.2) đảm bảo một ứng viên không ứng tuyển trùng lặp vào cùng một tin. Nếu vi phạm, backend bắt lỗi mã `P2002` của Prisma và trả về thông báo "Bạn đã ứng tuyển vào tin này rồi" thay vì để lỗi cơ sở dữ liệu lộ ra ngoài.

Hình 1.3 thể hiện trang chi tiết một tin tuyển dụng, nơi ứng viên bấm ứng tuyển.

Website kết nối việc làm thêm



Hình 1.3 thể hiện trang chi tiết một tin tuyển dụng, nơi ứng viên bấm ứng tuyển.

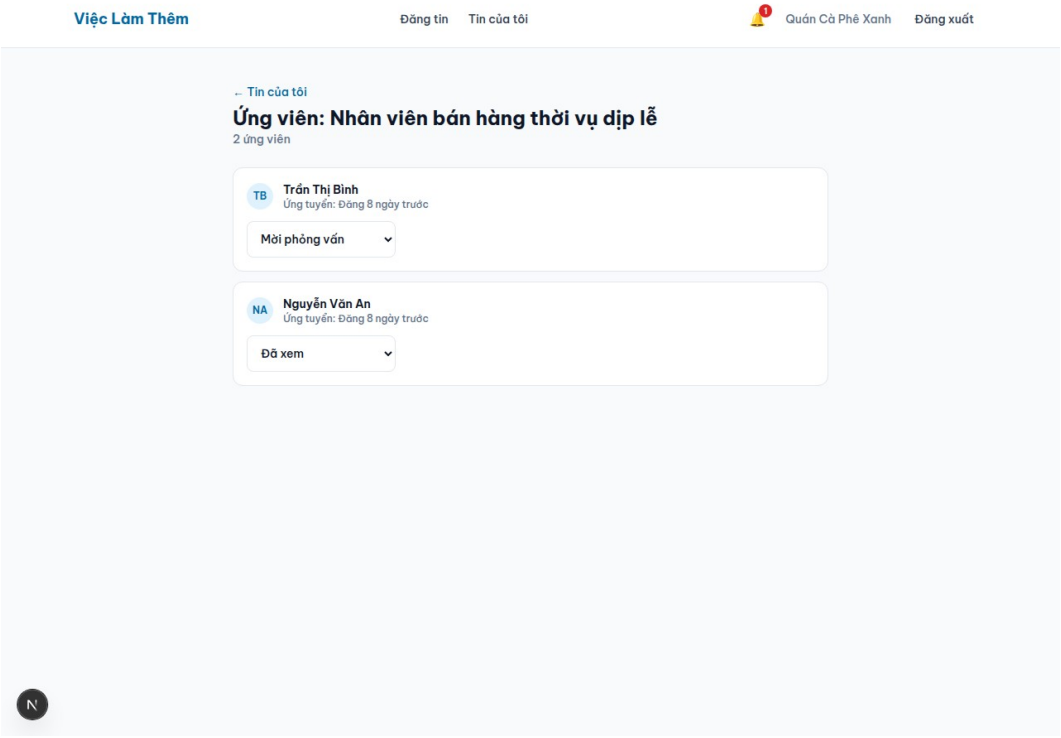
Nguồn: chụp từ hệ thống

3.6.5. UC05 — Quản lý danh sách ứng viên đã ứng tuyển

Nhà tuyển dụng xem danh sách ứng viên của một tin tại `/nha-tuyen-dung/tin/:id/ung-vien`, gọi `GET /jobs/:id/applications`. Lần gọi này đồng thời đánh dấu các hồ sơ đang ở trạng thái `PENDING` chuyển sang `VIEWED`. Nhà tuyển dụng chọn "Mời phỏng vấn" hoặc "Từ chối" để gọi `PATCH /applications/:id/status`, cập nhật lại `status` và tạo thông báo cho ứng viên (`backend/src/applications/applications.service.ts:102-130`, đã trích ở mục 3.4).

Hình 3.2 thể hiện trang quản lý ứng viên với hai hồ sơ ở hai trạng thái khác nhau.

Website kết nối việc làm thêm



Hình 3.2 thể hiện trang quản lý ứng viên với hai hồ sơ ở hai trạng thái khác nhau.

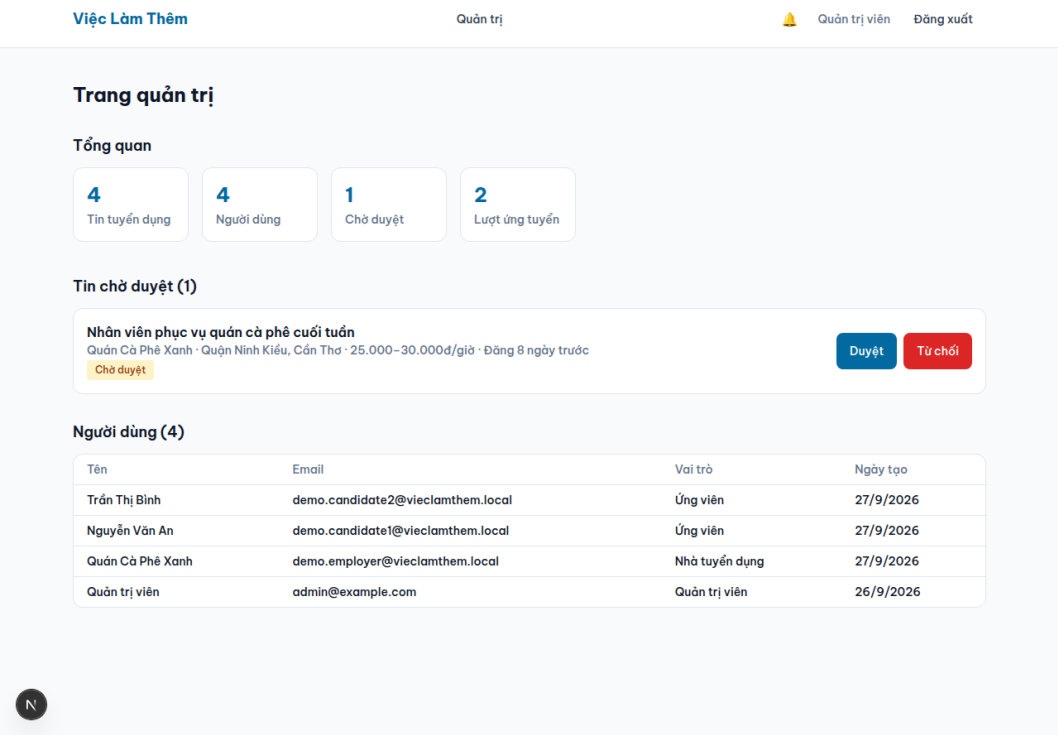
Nguồn: chụp từ hệ thống

3.6.6. UC06 — Duyệt hoặc từ chối tin tuyển dụng

Quản trị viên xem tin chờ duyệt tại `/quan-tri`, gọi `GET /admin/jobs` để lấy danh sách tin có `status = PENDING`, rồi gọi `PATCH /admin/jobs/:id/approve` hoặc `PATCH /admin/jobs/:id/reject` để chuyển trạng thái. Chỉ tin đã `APPROVED` mới hiển thị công khai và nhận được hồ sơ ứng tuyển (ràng buộc đã nêu ở mục 3.6.4).

Hình 3.3 thể hiện trang quản trị với tổng quan hệ thống, tin chờ duyệt và danh sách người dùng.

Website kết nối việc làm thêm



Hình 3.3 thể hiện trang quản trị với tổng quan hệ thống, tin chờ duyệt và danh sách người dùng.

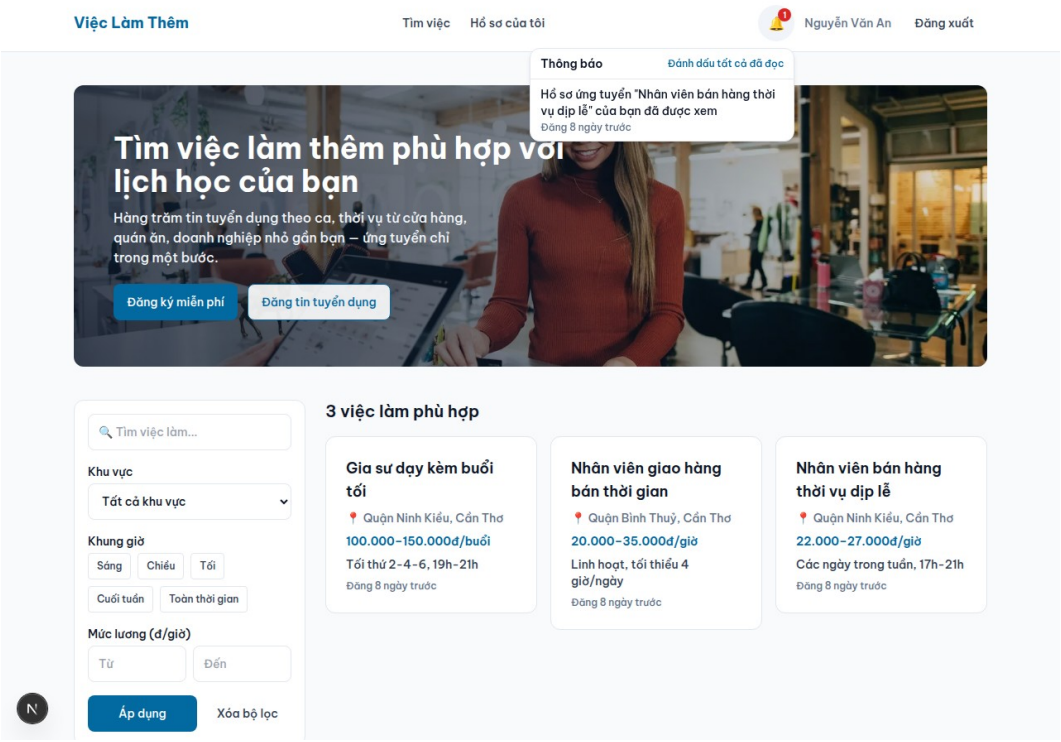
Nguồn: chụp từ hệ thống

3.6.7. UC07 — Nhận thông báo cập nhật hồ sơ

Cơ chế polling đã trình bày ở mục 2.6 và 3.1: mỗi 20 giây, giao diện gọi lại `GET /notifications/mine` và `GET /notifications/mine/unread-count` để cập nhật số thông báo chưa đọc trên biểu tượng chuông và danh sách thông báo khi mở ra.

Hình 3.4 thể hiện chuông thông báo đang mở, hiển thị một thông báo chưa đọc.

Website kết nối việc làm thêm



Hình 3.4 thể hiện chuông thông báo đang mở, hiển thị một thông báo chưa đọc.

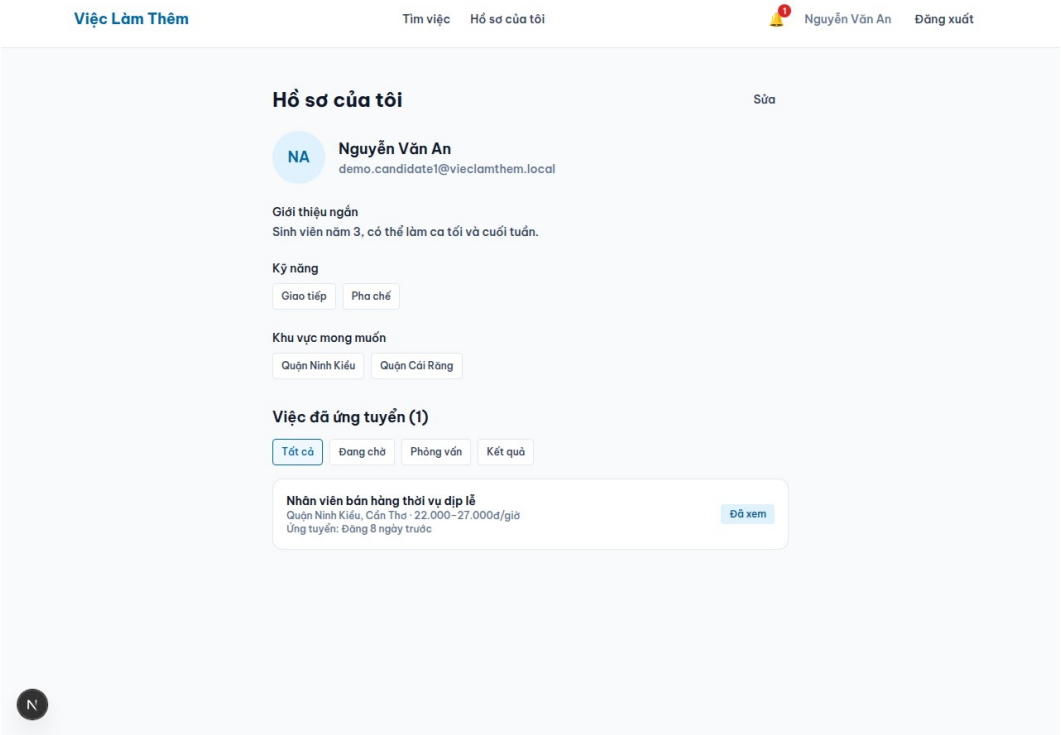
Nguồn: chụp từ hệ thống

3.7. Giao diện khác

Ngoài các màn hình đã minh họa theo từng use case ở trên, hệ thống còn có trang hồ sơ cá nhân của ứng viên.

Hình 3.5 thể hiện trang hồ sơ cá nhân của ứng viên (`/ho-so`), nơi ứng viên xem lại thông tin tài khoản của mình.

Website kết nối việc làm thêm



Hình 3.5 thể hiện trang hồ sơ cá nhân của ứng viên (`/ho-so`), nơi ứng viên xem lại thông tin tài khoản của mình.

Nguồn: chụp từ hệ thống

3.8. Xử lý lỗi và kiểm tra hợp lệ đầu vào

Cơ sở lý thuyết cho mục này đã trình bày ở mục 2.7 (nguyên tắc không tin dữ liệu từ client) và 2.8 (ràng buộc toàn vẹn ở tầng cơ sở dữ liệu). Mục này mô tả cụ thể cách hai nguyên tắc đó được hiện thực trong mã nguồn thật của hệ thống, cùng một ví dụ lỗi thật phát hiện được khi kiểm thử thủ công luồng đăng nhập.

Khi một DTO (mục 2.7) từ chối dữ liệu đầu vào, `ValidationPipe` của NestJS tự trả về mã trạng thái HTTP 400 kèm một mảng thông điệp lỗi liệt kê đúng những trường không hợp lệ, theo định dạng mặc định của framework, ví dụ với một yêu cầu đăng nhập thiếu cả email lẫn mật khẩu:

```
{
  "statusCode": 400,
  "message": ["email must be an email", "password must be longer than or equal to 1 characters"],
```

```
  "error": "Bad Request"
}
```

Phản hồi này được quan sát thật trong lúc kiểm thử thủ công luồng đăng nhập bằng Playwright khi dựng kịch bản ghi video demo (mục 4.5): kịch bản tự động điền form ngay sau khi điều hướng sang trang `/dang-nhap`, trước khi React kịp gắn trình xử lý sự kiện cho các ô nhập liệu (một tình huống tranh chấp thời gian — race condition — đặc thù của kiểm thử tự động, không xảy ra với người dùng thật vì luôn có độ trễ tự nhiên giữa lúc trang hiện ra và lúc họ bắt đầu gõ), khiến giá trị DOM được điền nhưng state của React vẫn rỗng, dẫn tới yêu cầu gửi đi với email/mật khẩu rỗng. `ValidationPipe` từ chối đúng yêu cầu rỗng này với thông điệp lỗi ở trên thay vì để lọt xuống tầng service và gây lỗi khó hiểu hơn ở sâu hơn trong hệ thống — đúng vai trò của một lớp kiểm tra đầu vào tập trung. Lỗi gốc (race condition trong kịch bản kiểm thử, không phải lỗi của form thật) được sửa bằng cách thêm một khoảng chờ ngắn sau khi điều hướng, trước khi điền form.

Ở phía front-end, mọi lời gọi API đều được bọc trong khối `try/catch`; khi nhận phản hồi lỗi (mã trạng thái từ 400 trở lên), giao diện đọc trường `message` trong phần thân phản hồi JSON và hiển thị trực tiếp nội dung đó thành thông báo cho người dùng (ví dụ dưới dạng toast hoặc dòng chữ đỏ cạnh ô nhập liệu sai), thay vì chỉ hiển thị một thông báo lỗi chung chung như "Có lỗi xảy ra" — cách làm này tận dụng lại đúng thông điệp lỗi chi tiết mà `ValidationPipe` đã sinh ra ở phía máy chủ, không phải viết lại một tập thông điệp lỗi riêng ở phía giao diện.

CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU

4.1. Kết quả kiểm thử tự động

Toàn bộ bộ kiểm thử được chạy lại tại thời điểm viết chương này (không dùng số liệu cũ), bằng `npm run test` và `npm run test:e2e` từ thư mục gốc repo:

```
Backend — kiểm thử đơn vị (npm run test --workspace=backend)
  Test Suites: 4 passed, 4 total
  Tests:       21 passed, 21 total

Backend — kiểm thử tích hợp/RBAC (npm run test:e2e --workspace=backend)
  Test Suites: 4 passed, 4 total
  Tests:       77 passed, 77 total

Frontend — kiểm thử đơn vị (npm run test --workspace=frontend)
  Test Suites: 2 passed, 2 total
  Tests:       8 passed, 8 total
```

Tổng cộng 106 kịch bản kiểm thử (21 + 77 + 8), toàn bộ đều đạt (pass), tỷ lệ 100%. Bốn bộ kiểm thử tích hợp bao phủ đúng bốn nhóm chức năng chính: xác thực (`auth.e2e-spec.ts`), tin tuyển dụng (`jobs.e2e-spec.ts`), hồ sơ ứng tuyển (`applications.e2e-spec.ts`), và phân quyền theo vai trò (`rbac.e2e-spec.ts`) — bộ kiểm thử RBAC riêng biệt này xác nhận một tài khoản không thể gọi được API dành cho vai trò khác (ví dụ ứng viên gọi API duyệt tin của quản trị viên sẽ bị từ chối với mã lỗi 403), đúng yêu cầu phi chức năng "không có truy cập chéo vai trò khi chưa đăng nhập đúng quyền" đặt ra từ đầu.

4.2. Đối chiếu với chỉ tiêu định lượng đã đề ra

Bảng 4.1 đối chiếu kết quả thật với các chỉ tiêu định lượng đã đề xuất trước khi xây dựng hệ thống.

Bảng 4.1. Đối chiếu chỉ tiêu định lượng

Chỉ tiêu	**Mức đề xuất**	**Kết quả thật**	**Đạt/Chưa đạt**
Số use case hoàn thành (trên 7 use	Tối thiểu 6/7	7/7	Đạt, vượt chỉ tiêu

Chỉ tiêu	Mức đề xuất	Kết quả thật	Đạt/Chưa đạt
case)			
Thời gian tải trang chủ (môi trường demo)	Không quá 3 giây	Khoảng 15-190 mili-giây	Đạt
Tỷ lệ kịch bản kiểm thử đạt	Từ 90% trở lên	106/106 (100%)	Đạt
Khả năng sử dụng trên điện thoại	Hiển thị đúng, thao tác được	Không tràn ngang ở 3 độ rộng màn hình 360/768/1280px (đã kiểm bằng Chromium thật)	Đạt

Thời gian tải trang và kết quả kiểm tra tràn ngang màn hình được đo trong giai đoạn xây dựng hệ thống (xem phần "Completion Summary" của kế hoạch triển khai dự án), không đo lại trong phạm vi viết báo cáo này vì không có thay đổi nào ở giao diện trang chủ hay CSS responsive kể từ đó.

4.3. Một số nhận xét về giao diện

Giao diện được tổ chức nhất quán cho ba vai trò: mỗi vai trò có một thanh điều hướng riêng chỉ hiển thị đúng chức năng vai trò đó được phép dùng (ứng viên không thấy mục "Đăng tin", nhà tuyển dụng không thấy mục "Quản trị"), giảm nguy cơ người dùng bấm nhầm vào chức năng không thuộc về mình. Thao tác ứng tuyển và cập nhật trạng thái hồ sơ chỉ cần một lượt bấm và chọn trong danh sách sổ xuống, không cần chuyển qua nhiều trang trung gian.

4.4. Hạn chế đã biết

Một số giới hạn được ghi nhận rõ ràng, không che giấu:

- `docker compose up --build` được viết đúng cấu hình và đã kiểm tra bằng `docker compose

config`, nhưng chưa được chạy end-to-end trong môi trường không có Docker daemon khả dụng lúc viết báo cáo này.

- Thông báo cập nhật có độ trễ tối đa khoảng 20 giây do dùng cơ chế polling thay vì đẩy dữ liệu

tức thời (đã giải thích lý do lựa chọn ở mục 2.6).

- Bộ dữ liệu kiểm thử/demo hiện tại còn ở quy mô nhỏ (vài tài khoản, vài tin tuyển dụng), chưa

phản ánh tải thật của một hệ thống có nhiều người dùng đồng thời.

- Phần hiển thị chuông thông báo và vòng lặp polling ở giao diện (UC07) chưa có kiểm thử front-end

riêng — chỉ được xác nhận gián tiếp qua kiểm thử back-end rằng thông báo được tạo đúng trong cơ sở dữ liệu (xem mục 4.5), chưa kiểm tra phần hiển thị thật trên giao diện.

4.5. Phạm vi kiểm thử theo nhóm chức năng

Bảng 4.2 đối chiếu 7 use case đã đặc tả (mục 3.6) với các bộ kiểm thử tự động thực sự bao phủ từng use case đó, dựa trực tiếp trên tên file kiểm thử trong `backend/src/` và `backend/test/` (mục 2.9 trình bày cơ sở lý thuyết cho sự phân tầng kiểm thử đơn vị/tích hợp này).

Bảng 4.2. Đối chiếu use case với bộ kiểm thử tự động

Use case	**Kiểm thử đơn vị**	**Kiểm thử tích hợp (e2e)**
UC01 — Đăng ký/đăng nhập	`auth.service.spec.ts`	`auth.e2e-spec.ts`
UC02 — Đăng tin tuyển dụng	`jobs.service.spec.ts`	`jobs.e2e-spec.ts`

Use case	**Kiểm thử đơn vị**	**Kiểm thử tích hợp (e2e)**
UC03 — Tìm kiếm và lọc tin	`jobs.service.spec.ts`	`jobs.e2e-spec.ts`
UC04 — Ứng tuyển vào tin	`applications.service.spec.ts`	`applications.e2e-spec.ts`
UC05 — Quản lý ứng viên	`applications.service.spec.ts`	`applications.e2e-spec.ts`
UC06 — Duyệt/từ chối tin	`jobs.service.spec.ts`	`jobs.e2e-spec.ts`
UC07 — Nhận thông báo	—	gián tiếp qua `applications.e2e-spec.ts` (xem ghi chú)
Mọi use case — phân quyền chéo vai trò	`roles.guard.spec.ts`	`rbac.e2e-spec.ts`

Ghi chú: UC07 (thông báo) không có file kiểm thử riêng vì bản thân endpoint `/notifications/mine` chỉ là một truy vấn đọc đơn giản. Phạm vi bao phủ của nó nằm gián tiếp trong `applications.e2e-spec.ts`: một trong các kịch bản ở đó, sau khi gọi `PATCH /applications/:id/status`, gọi tiếp `GET /notifications/mine` và xác nhận có một thông báo kiểu `APPLICATION_STATUS_CHANGED` xuất hiện — xác nhận đúng luồng "cập nhật trạng thái → tạo thông báo" mô tả ở mục 3.4, nhưng không kiểm tra riêng phần hiển thị chuông thông báo hay vòng lặp polling ở giao diện (chưa có kiểm thử front-end nào bao phủ hai phần này — một hạn chế thật, bổ sung vào mục 4.4). Việc một use case nghiệp vụ (UC02, UC03, UC06) có thể cùng được bao phủ bởi một file kiểm thử chung (`jobs.service.spec.ts`, `jobs.e2e-spec.ts`) là hợp lý vì cả ba đều thao tác trên cùng một service/controller `Job`, chỉ khác hành động cụ thể (tạo tin, tìm kiếm, duyệt tin).

4.6. Đánh giá khả năng mở rộng hệ thống

Ba lựa chọn kiến trúc đã trình bày ở Chương 2-3 có ảnh hưởng trực tiếp tới khả năng mở rộng (scalability) của hệ thống nếu lượng người dùng tăng lên vượt quy mô demo hiện tại. Xác thực bằng JWT (mục 2.5.1) là cơ chế không trạng thái (stateless): máy chủ không cần lưu phiên đăng nhập trong bộ nhớ hay cơ sở dữ liệu, toàn bộ thông tin cần để xác thực một yêu cầu đã nằm sẵn trong chính token gửi kèm. Nhờ vậy, dịch vụ `backend` có thể chạy nhiều bản sao song song sau một bộ cân bằng tải (load balancer) mà không cần đồng bộ trạng thái phiên giữa các bản sao đó — một yêu cầu đăng nhập ở bản sao A và một yêu cầu tiếp theo được định tuyến tới bản sao B vẫn xác thực đúng, vì cả hai chỉ cần cùng một khóa bí mật (`JWT_SECRET`) để xác minh chữ ký token, không cần tra cứu lại một bảng phiên tập trung.

Ngược lại, cơ chế polling thông báo (mục 2.6) là điểm có khả năng mở rộng kém nhất trong hệ thống hiện tại: mỗi client đang mở ứng dụng gọi lại API cứ mỗi 20 giây bất kể có thông báo mới hay không, nên tải lên `backend` tăng tuyến tính theo số người dùng đang hoạt động đồng thời, không theo số sự kiện thông báo thực sự xảy ra. Ở quy mô vài chục người dùng như môi trường demo của đồ án, mức tải này không đáng kể; nhưng nếu số người dùng đồng thời tăng lên hàng nghìn, tần suất gọi API cố định này sẽ trở thành điểm nghẽn trước khi các phần còn lại của hệ thống đạt giới hạn, đúng như đã nêu ở mục 2.6 khi so sánh với phương án WebSocket.

Về tầng dữ liệu, Prisma Client mặc định quản lý một connection pool kết nối tới PostgreSQL thay vì mở một kết nối mới cho mỗi truy vấn, giúp giảm chi phí thiết lập kết nối khi tải tăng; tuy nhiên cấu hình kích thước pool hiện vẫn dùng giá trị mặc định của Prisma, chưa được đo đạc và điều chỉnh theo tải thật — đây là một điểm cần khảo sát thêm nếu hệ thống được đưa vào vận hành với lượng người dùng lớn hơn đáng kể so với quy mô một đồ án cơ sở ngành.

4.7. Đối chiếu kết quả với hiện trạng ban đầu

Mục 1.2 và 1.3 (Chương 1) đã khảo sát hai nhóm giải pháp hiện có trước khi xây dựng hệ thống: các nền tảng tuyển dụng lớn (TopCV, VietnamWorks, JobStreet) nặng về quy trình và không tối ưu cho việc làm thêm ngắn hạn, còn các kênh phi chính thức (nhóm

Facebook, Zalo) thiếu công cụ tìm kiếm/lọc và không xác thực người đăng tin. Bảng 4.3 đối chiếu lại ba hạn chế cụ thể đã nêu ra ở Chương 1 với cách hệ thống thật sự đã xây dựng giải quyết từng hạn chế đó.

Bảng 4.3. Đối chiếu hạn chế ban đầu với giải pháp đã cài đặt

Hạn chế nêu ở Chương 1	**Giải pháp đã cài đặt**	**Vị trí trong báo cáo**
Tin đăng rải rác nhiều nhóm, khó tìm, dễ trôi	Một kho tin tập trung, tìm kiếm/lọc theo `q`, `area`, `shift`, mức lương qua `GET /jobs`	Mục 3.3, UC03 (3.6.3)
Không có bộ lọc theo khu vực/khung giờ/mức lương	Bộ lọc kết hợp nhiều tiêu chí cùng lúc, cài đặt thật và kiểm thử qua `jobs.e2e-spec.ts`	Mục 3.3, 4.5
Không xác thực người đăng tin, dễ xuất hiện tin giả	Bắt buộc đăng ký tài khoản có mật khẩu băm bcrypt (mục 2.5.2) trước khi đăng tin; mọi tin mới đều ở trạng thái `PENDING`, chỉ hiển thị công khai sau khi quản trị viên duyệt (UC06, 3.6.6)	Mục 2.5, 3.6.6

Việc đối chiếu này cho thấy cả ba hạn chế cụ thể nêu ra ở phần khảo sát hiện trạng đều có một giải pháp cài đặt thật tương ứng, không chỉ dừng ở mức đề xuất trên giấy — nhất quán với cách tiếp cận lặp đã mô tả ở mục 1.5 (thiết kế, cài đặt và kiểm thử ngay sau khi hoàn thành từng nhóm chức năng, thay vì dồn toàn bộ kiểm thử về cuối).

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đồ án đã xây dựng hoàn chỉnh một website kết nối người tìm việc làm thêm với nhà tuyển dụng, đạt đủ 6 mục tiêu cụ thể đã đề ra ở phần Mở đầu: khảo sát yêu cầu hai nhóm người dùng, thiết kế cơ sở dữ liệu quan hệ 5 bảng, xây dựng API RESTful cho toàn bộ nghiệp vụ, xây dựng giao diện cho ba vai trò, cài đặt tìm kiếm/lọc/thông báo, và kiểm thử hệ thống với một bản demo chạy được.

Toàn bộ 7 use case đã đặc tả đều được cài đặt và kiểm thử, vượt chỉ tiêu tối thiểu 6/7 đã đề ra ban đầu. Kết quả kiểm thử tự động đạt 106/106 kịch bản, vượt chỉ tiêu tối thiểu 90%. Hệ thống giải quyết đúng khoảng trống thị trường đã nêu ở Chương 1: một kênh vừa đủ nhẹ để đăng tin nhanh như nhóm mạng xã hội, vừa có công cụ tìm kiếm/lọc và một bước kiểm duyệt tối thiểu mà các nền tảng tuyển dụng lớn không tối ưu cho quy mô việc làm thời vụ, ngắn hạn.

Đóng góp cụ thể của đồ án nằm ở ba điểm: một hệ thống demo chuyên biệt cho việc làm thêm, khác với các nền tảng tuyển dụng việc làm toàn thời gian hiện có; bộ lọc theo khu vực, khung giờ và mức lương phù hợp đặc thù công việc bán thời gian; và một bộ mã nguồn cùng tài liệu thiết kế có thể tái sử dụng hoặc mở rộng cho các đồ án, dự án tương tự sau này.

Bên cạnh kết quả đạt được, đồ án còn tồn tại các hạn chế đã nêu ở mục 4.4, chủ yếu liên quan đến quy mô kiểm thử còn nhỏ và độ trễ của cơ chế thông báo — đây cũng chính là cơ sở cho các hướng phát triển tiếp theo.

5.2. Hướng phát triển

- Xây dựng ứng dụng di động (mobile app) riêng cho hai vai trò ứng viên và nhà tuyển dụng, tận

dụng lại toàn bộ API RESTful đã có sẵn.

- Bổ sung gợi ý việc làm theo lịch sử tìm kiếm của ứng viên, thay vì chỉ lọc theo tiêu chí người

dùng tự chọn như hiện tại.

- Tích hợp xác thực danh tính (ví dụ đối chiếu thông tin cơ bản) để tăng độ tin cậy giữa hai bên

trước khi nhận việc, mà vẫn giữ quy trình đăng ký đơn giản như đã thiết kế.

- Bổ sung cơ chế đánh giá hai chiều giữa ứng viên và nhà tuyển dụng sau khi kết thúc một đợt làm

việc, giúp tích lũy uy tín cho cả hai phía qua thời gian.

- Thay cơ chế polling bằng kết nối đẩy dữ liệu tức thời (WebSocket) nếu quy mô người dùng tăng

đủ lớn để độ trễ 20 giây trở thành vấn đề thực sự, đúng như đã cân nhắc ở mục 2.6.

- Triển khai bản demo công khai bằng Docker Compose trên một máy chủ thật, hoàn tất phần còn

thiếu đã nêu ở mục 4.4.

- Thu hẹp cấu hình CORS về đúng domain thật và bổ sung giới hạn số lần thử đăng nhập sai (rate

limiting, ví dụ bằng `@nestjs/throttler`) trước khi đưa hệ thống ra khỏi môi trường demo — hai điểm đã nêu là đánh đổi phạm vi có chủ đích ở mục 2.11.

DANH MỤC TÀI LIỆU THAM KHẢO

Danh mục dưới đây chỉ gồm các tài liệu thực sự được trích dẫn trong Chương 1-3, sắp xếp theo thứ tự từ điển (họ tác giả hoặc tên tổ chức đầu mục) theo đúng quy định của khoa, đánh số `[1]`-`[18]` theo vị trí đó — số thứ tự này KHÔNG theo thứ tự xuất hiện trong bài. 4 tài liệu mới ([3], [4], [5], [16]) bổ sung ngày 06/10 khi viết thêm mục 2.7-2.10 và 3.8, chèn đúng vị trí từ điển vào danh mục 14 tài liệu cũ — mọi số thứ tự cũ từ vị trí chèn trở đi đã dịch lại và cập nhật khớp trong Chương 1-2 (không có tài liệu nào mất trích dẫn hoặc trích dẫn sai số).

[1] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, Dec. 2011.

[2] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[3] M. Fowler, "TestPyramid," martinfowler.com, May 2012. [Online]. Available: https://martinfowler.com/bliki/TestPyramid.html

[4] GraphQL Foundation, "GraphQL Documentation," [Online]. Available: https://graphql.org/learn

[5] gRPC Authors, "Introduction to gRPC," gRPC Documentation, [Online]. Available: https://grpc.io/docs/what-is-grpc/introduction

[6] Instawork, "Instawork — Flexible shift work," [Online]. Available: https://www.instawork.com

[7] JobStreet, "JobStreet Vietnam," [Online]. Available: https://www.jobstreet.vn

[8] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[9] NestJS, "NestJS – A progressive Node.js framework," NestJS Documentation, [Online]. Available: https://docs.nestjs.com

[10] Next.js, "Next.js Documentation," Vercel, [Online]. Available: https://nextjs.org/docs

[11] OWASP Foundation, "OWASP Top 10:2021 – The ten most critical web application security risks," 2021. [Online]. Available: https://owasp.org/Top10

[12] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs

[13] Prisma, "Prisma ORM Documentation," 2024. [Online]. Available: https://www.prisma.io/docs

[14] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, 1999.

[15] TopCV, "TopCV — Tìm việc làm nhanh," [Online]. Available: https://www.topcv.vn

[16] typestack, "class-validator," GitHub, [Online]. Available: https://github.com/typestack/class-validator

[17] VietnamWorks, "VietnamWorks," [Online]. Available: https://www.vietnamworks.com

[18] Wonolo, "Wonolo — On-demand staffing," [Online]. Available: https://www.wonolo.com

PHỤ LỤC A. TOÀN VĂN SCHEMA CƠ SỞ DỮ LIỆU

Nội dung dưới đây copy nguyên văn từ `backend/prisma/schema.prisma` tại thời điểm viết báo cáo.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role {
  CANDIDATE
  EMPLOYER
  ADMIN
}

enum JobStatus {
  PENDING
  APPROVED
  REJECTED
}

enum ApplicationStatus {
  PENDING
  VIEWED
  INTERVIEW
  REJECTED
}

model User {
  id           String   @id @default(uuid())
  email        String   @unique
  passwordHash String
  role         Role
  fullName     String
  phone        String?
```

```
  isActive     Boolean  @default(true)
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  profile       Profile?
  jobs          Job[]           @relation("EmployerJobs")
  applications  Application[]
  notifications Notification[]
}

model Job {
  id           String    @id @default(uuid())
  title        String
  description  String
  area         String
  shift        String
  salaryMin    Int?
  salaryMax    Int?
  salaryUnit   String    @default("VND/giờ")
  requirements String?
  status       JobStatus @default(PENDING)
  rejectReason String?
  employerId   String
  employer     User      @relation("EmployerJobs", fields: [employerId],
references: [id])
  applications Application[]
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt

  @@index([status, area])
}

model Application {
  id          String            @id @default(uuid())
  jobId       String
  job         Job               @relation(fields: [jobId], references: [id])
  candidateId String
  candidate   User              @relation(fields: [candidateId], references:
[id])
  status      ApplicationStatus @default(PENDING)
```

```
  message     String?
  createdAt   DateTime          @default(now())
  updatedAt   DateTime          @updatedAt

  @@unique([jobId, candidateId])
}

model Profile {
  id             String   @id @default(uuid())
  userId         String   @unique
  user           User     @relation(fields: [userId], references: [id])
  bio            String?
  skills         String[] @default([])
  preferredAreas String[] @default([])
  avatarUrl      String?
}

model Notification {
  id        String   @id @default(uuid())
  userId    String
  user      User     @relation(fields: [userId], references: [id])
  type      String
  message   String
  link      String?
  isRead    Boolean  @default(false)
  createdAt DateTime @default(now())

  @@index([userId, isRead])
}
```

PHỤ LỤC B. TOÀN VĂN DANH SÁCH API

Bảng dưới đây liệt kê đầy đủ mọi endpoint thật trong `backend/src/`, đối chiếu trực tiếp với từng file `*.controller.ts` — không suy đoán route chưa xác nhận trong code.

Phương thức	**Endpoint**	**File controller**	**Vai trò yêu cầu**
GET	`/health`	health/ health.controller.ts	Công khai
POST	`/auth/register`	auth/ auth.controller.ts	Công khai
POST	`/auth/login`	auth/ auth.controller.ts	Công khai
GET	`/auth/me`	auth/ auth.controller.ts	Đã đăng nhập
POST	`/jobs`	jobs/ jobs.controller.ts	EMPLOYER
GET	`/jobs/mine`	jobs/ jobs.controller.ts	EMPLOYER
GET	`/jobs`	jobs/ jobs.controller.ts	Công khai
GET	`/jobs/:id`	jobs/ jobs.controller.ts	Công khai (có nhận diện người xem nếu đã đăng nhập)
PATCH	`/jobs/:id`	jobs/ jobs.controller.ts	EMPLOYER, chỉ chủ tin

Phương thức	**Endpoint**	**File controller**	**Vai trò yêu cầu**
			(`JobOwnerGuard`)
DELETE	`/jobs/:id`	jobs/ jobs.controller.ts	EMPLOYER, chỉ chủ tin (`JobOwnerGuard`)
POST	`/applications`	applications/ applications.controller.ts	CANDIDATE
GET	`/applications/mine`	applications/ applications.controller.ts	CANDIDATE
GET	`/jobs/:id/ applications`	applications/ applications.controller.ts	EMPLOYER, chỉ chủ tin (`JobOwnerGuard`)
PATCH	`/applications/:id/ status`	applications/ applications.controller.ts	EMPLOYER
GET	`/notifications/ mine`	notifications/ notifications.controller.ts	Đã đăng nhập
GET	`/notifications/ mine/unread-count`	notifications/ notifications.controller.ts	Đã đăng nhập
PATCH	`/notifications/:id/ read`	notifications/ notifications.control	Đã đăng nhập

Phương thức	**Endpoint**	**File controller**	**Vai trò yêu cầu**
		ler.ts	
PATCH	`/notifications/read-all`	notifications/ notifications.controller.ts	Đã đăng nhập
GET	`/profiles/me`	profiles/ profiles.controller.ts	CANDIDATE
PATCH	`/profiles/me`	profiles/ profiles.controller.ts	CANDIDATE
GET	`/admin/jobs`	admin/admin-jobs.controller.ts	ADMIN
PATCH	`/admin/jobs/:id/approve`	admin/admin-jobs.controller.ts	ADMIN
PATCH	`/admin/jobs/:id/reject`	admin/admin-jobs.controller.ts	ADMIN
GET	`/admin/users`	admin/admin-users.controller.ts	ADMIN
GET	`/admin/stats`	admin/admin-stats.controller.ts	ADMIN

Tổng cộng 24 endpoint (không tính `/health`), phân bổ trên 8 controller.